Introduction to Model Deployment

Learning Objectives

By the end of this session, you should be able to:

- Understand the need for model deployment and its role in operationalizing Machine Learning solutions.
- Implement serialization techniques to efficiently store and transfer trained models.
- Develop API endpoints to facilitate seamless interaction with deployed models.
- Manage dependencies and hosting environments to ensure reliable and scalable model deployment.

Agenda

In this session, we'll discuss:

- The Need for Model Deployment
- Introduction to Model Deployment
- Serialization
- Introduction to APIs
- Endpoints
- Handling Dependencies
- Hosting a Deployed Model
- Architecture of Model Deployment

Video Break

Common Problems Solved with Machine Learning

Customer Churn Prediction

Predicting which customers are likely to churn based on usage patterns and complaints

Common Problems Solved with Machine Learning

Customer Churn Prediction	Predicting which customers are likely to churn based on usage patterns and complaints
Predictive Maintenance	Identifying machinery components likely to fail before they actually do based on sensor data

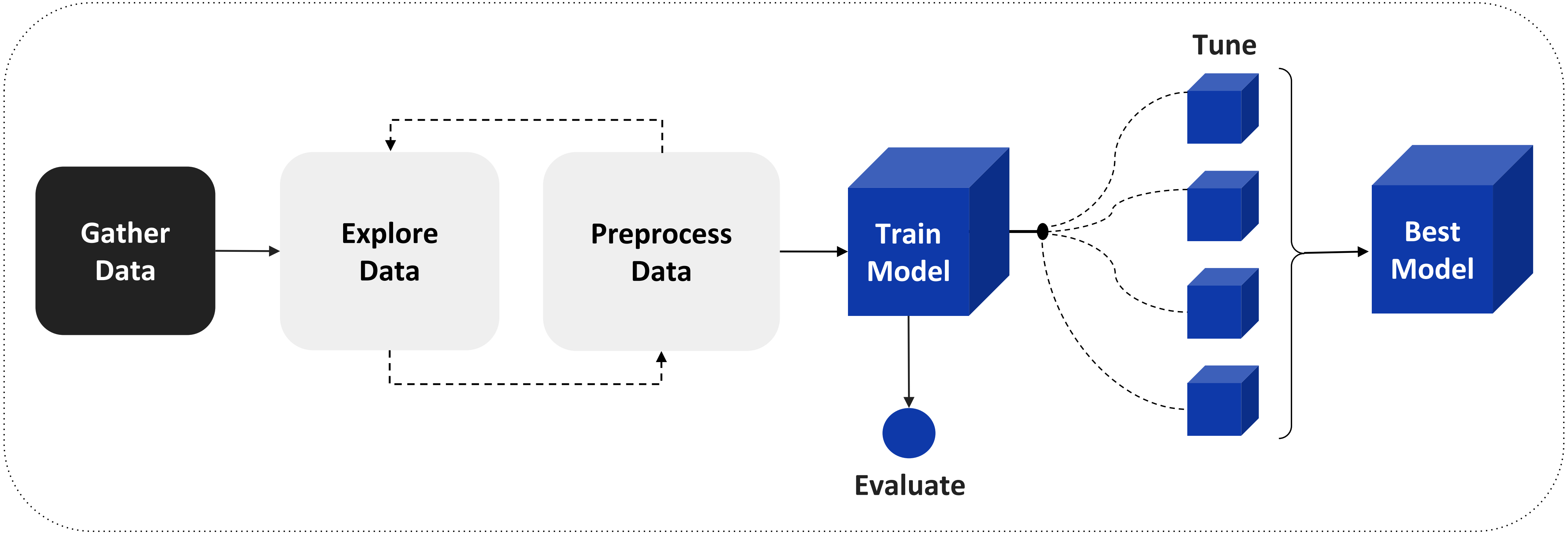
Common Problems Solved with Machine Learning

Customer Churn Prediction	**Predicting which customers are likely to churn based on usage patterns and complaints**
Predictive Maintenance	**Identifying machinery components likely to fail before they actually do based on sensor data**
Fraud Detection	**Detecting fraudulent transactions based on historical transaction data**

Common Problems Solved with Machine Learning

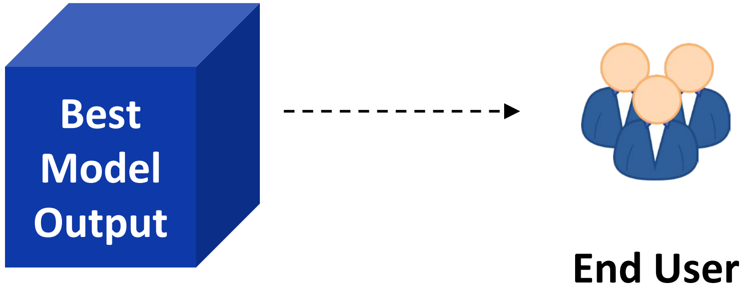
Customer Churn Prediction	Predicting which customers are likely to churn based on usage patterns and complaints
Predictive Maintenance	Identifying machinery components likely to fail before they actually do based on sensor data
Fraud Detection	Detecting fraudulent transactions based on historical transaction data
Personalised Recommendations	Suggesting products or content based on user demography and consumption patterns

Machine Learning Model Building Process



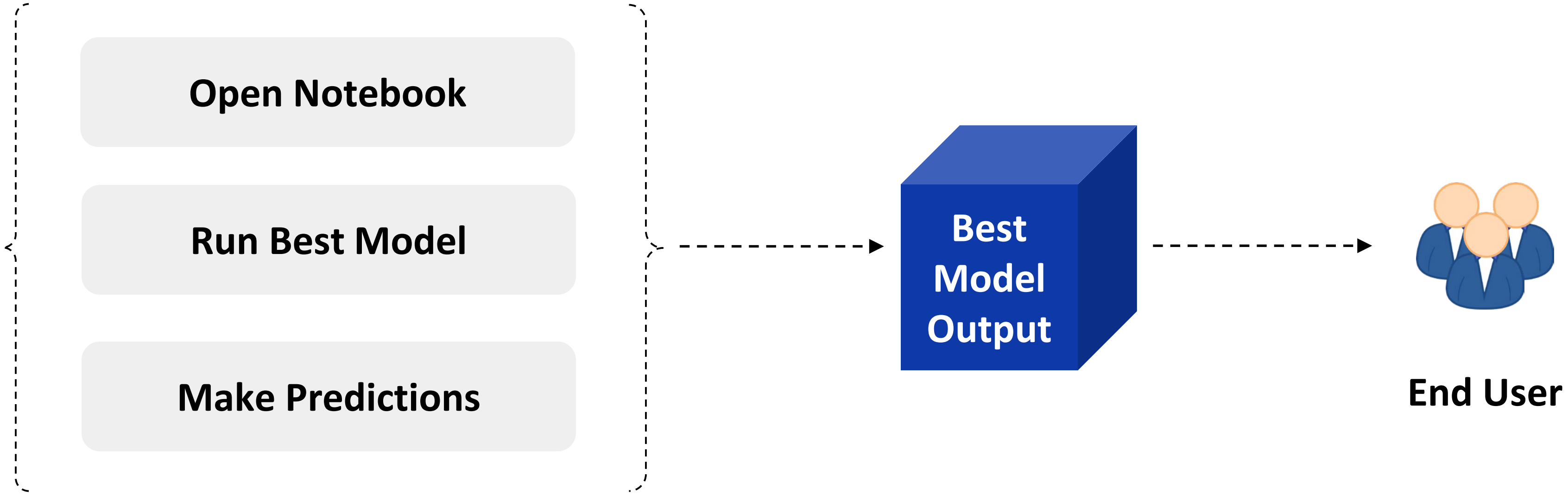
Model Output Consumption

- Need to **expose the best model's output to the end user** for consumption.
- How do we do this?



Model Output Consumption

- Need to **expose the best model's output to the end user** for consumption.
- A **simple approach** can be to **share the Python notebook** with the end user.



Challenges with Sharing Python Notebooks

- Consider the example of a content recommendation model for an online streaming service trained on user watch history.

Accessibility

- How will end users **access the best model** from the data scientist's Python notebook?
- The Python notebook is a set of codes and the end user might not necessarily have the relevant expertise

Users need a **simpler method** to **receive recommendations** as per their preferences without being exposed to associated technicalities.

Challenges with Sharing Python Notebooks

- Consider the example of a content recommendation model for an online streaming service trained on user watch history.

Accessibility

Configuration

- The best model relies on specific dependencies (libraries, packages, and more).
- How do we ensure that the **same "setup" is available on the user's end**?

Even if a user streams from different devices, we **need to ensure dependencies are taken care for smoothly functionality**.

Challenges with Sharing Python Notebooks

- Consider the example of a content recommendation model for an online streaming service trained on user watch history.

Accessibility

Configuration

Performance

- The best model must analyze large amounts of data to identify patterns and generate relevant output.
- Will the Python notebook on the data scientist's system be able to **generate recommendations fast enough**?

If the **recommendations aren't shown at the right time**, then **user engagement will reduce** and consequently impact revenue.

Challenges with Sharing Python Notebooks

- Consider the example of a content recommendation model for an online streaming service trained on user watch history.

Accessibility

Configuration

Performance

Scalability

- The best model is in a single notebook on the data scientist's local system.
- How will it **handle the requirements of multiple consumers**, each expecting instant recommendations?

If the model **cannot provide recommendations to multiple users**, it can lead to a **suboptimal user experience** for those left out, resulting in lower retention rates.

Video Break

Addressing Notebook Sharing Challenges

- To make the model accessible and usable, we need:

1	2	3	4
A way to **share** the model	A way to **ensure consistent configurations**	**Interface for communication** to get fast predictions	**Concurrent access** for multiple users

Addressing Notebook Sharing Challenges

- To make the model accessible and usable, we need:

A way to **share** the model

A way to **ensure consistent configurations**

Interface for communication to get fast predictions

Concurrent access for multiple users

Sharing and Consistency

“Package” the model and dependencies

Speed and Scalability

“Set” it up for **fast** and **concurrent “usage”**

Addressing Notebook Sharing Challenges

- To make the model accessible and usable, we need:

A way to **share** the model

A way to **ensure consistent configurations**

Interface for communication to get fast predictions

Concurrent access for multiple users

Sharing and Consistency

“Package” the model and dependencies

Speed and Scalability

“Set” it up for **fast** and **concurrent “usage”**

That's where **model deployment** comes in.

Model Deployment

Deploying a model means taking a trained ML model, **packaging** it, and setting it up for **inference**.

Model Deployment

Deploying a model means taking a trained ML model, **packaging** it, and setting it up for **inference**.

- Note that we **directly take a trained model**. We **DON'T train it during deployment**.
- **Inference** is the process of inputting data points into a **machine learning model** to generate an output.

Model Deployment

Deploying a model means taking a trained ML model, **packaging** it, and setting it up for **inference**.

- Note that we **directly take a trained model**. We **DON'T train it during deployment**.
- **Inference** is the process of inputting data points into a **machine learning model** to generate an output.

How do we "**package**"? **1**

Model Deployment

Deploying a model means taking a trained ML model, **packaging** it, and setting it up for **inference**.

- Note that we **directly take a trained model**. We **DON'T train it during deployment**.
- **Inference** is the process of inputting data points into a **machine learning model** to generate an output.

1. How do we "**package**"?
2. How do we "**set up**" for **inference**?

Video Break

Addressing Notebook Sharing Challenges

A way to **share** the model

1

A way to **ensure consistent configurations**

2

Interface for communication to get fast predictions

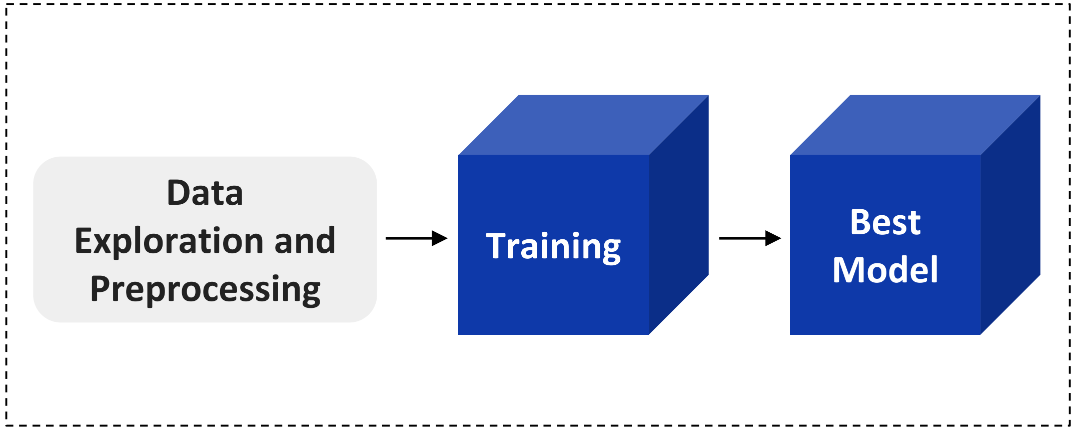
3

Concurrent access for multiple users

4

- Let's start with how we can "**share**" **the best model**.

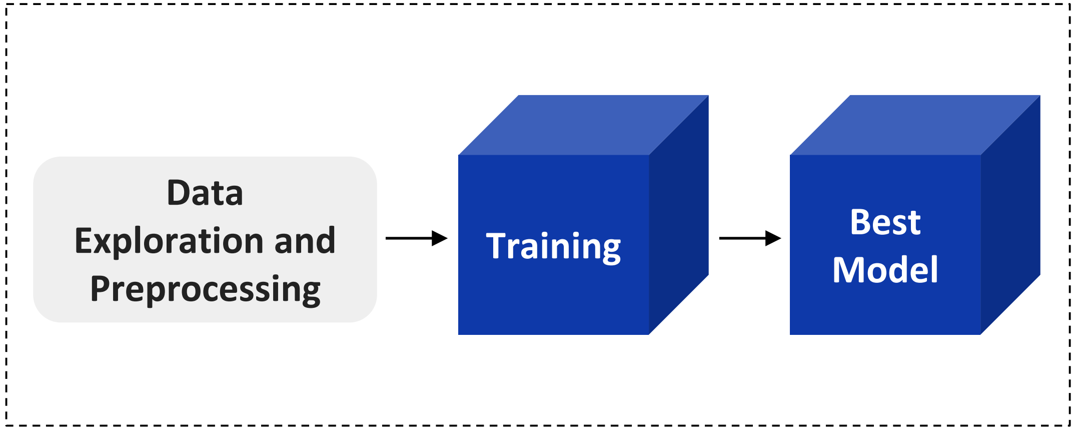
Current Setup



Local System

- Recall that all the steps involved in the model building process were being executed in a Python notebook on the data scientist's local system.

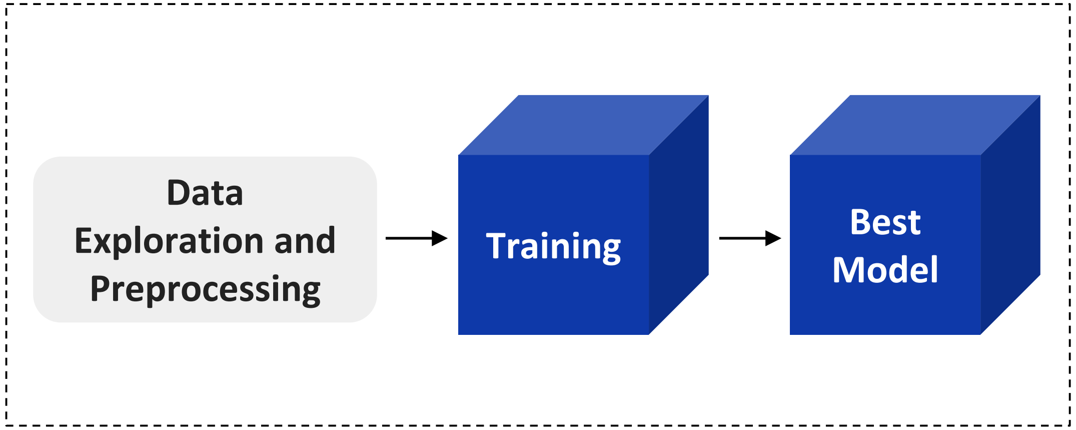
Current Setup



Local System

- Recall that all the steps involved in the model building process were being executed in a Python notebook on the data scientist's local system.
- The notebook can be created in any IDE (Integrated Development Environment).

Current Setup



Local System

- Recall that all the steps involved in the model building process were being executed in a Python notebook on the data scientist's local system.
- The notebook can be created in any IDE (Integrated Development Environment).
- Depending on the IDE used, there'll be a specific configuration required to execute the relevant code.

This configuration under which the model was built is an **environment**.

Environment

An **environment** is a **specific set of conditions and configurations** under which software and systems operate.

Environment

An **environment** is a **specific set of conditions and configurations** under which software and systems operate.

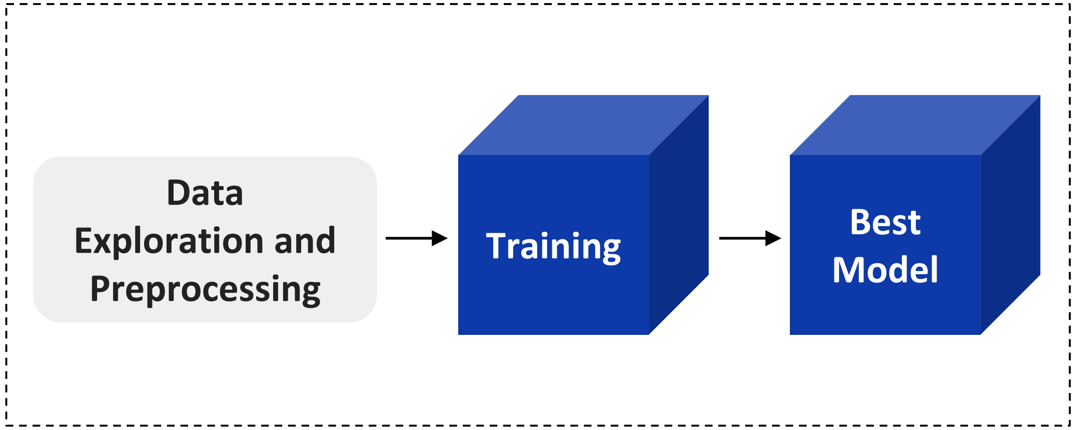
Development Environment

Primarily used for training ML models by hyperparameter tuning.

Production Environment

Primarily used to deploy the final ML model to serve end users.

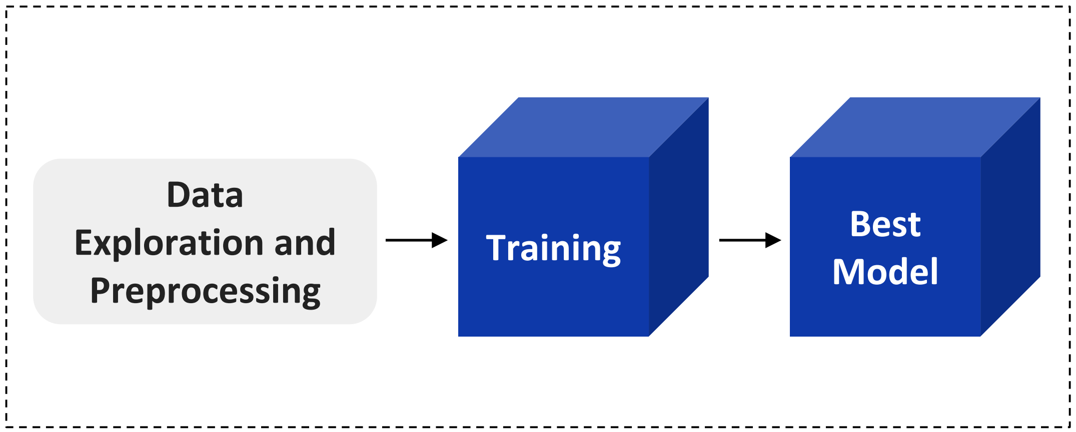
The Need for Serialization



Development Environment

- The best model is in the development environment and we need to make predictions for new data.

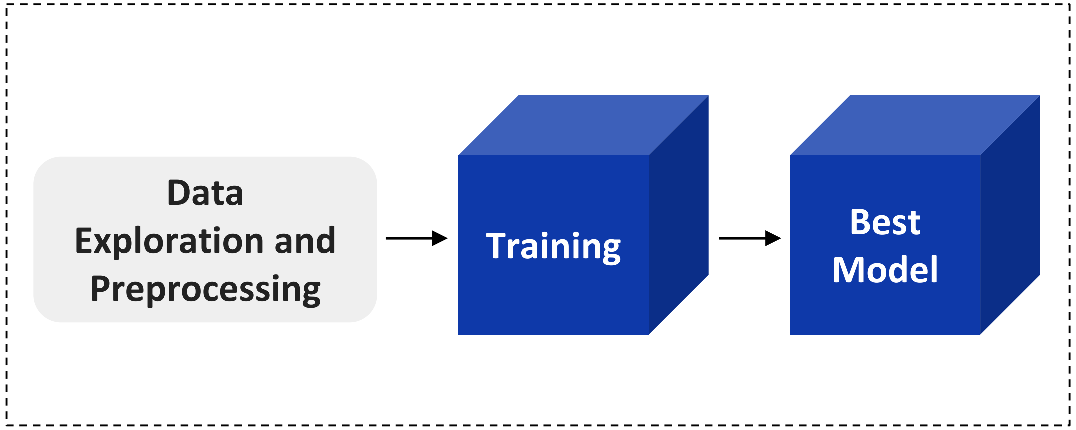
The Need for Serialization



Development Environment

- The best model is in the development environment and we need to make predictions for new data.
- What if we **shut down** the development environment?

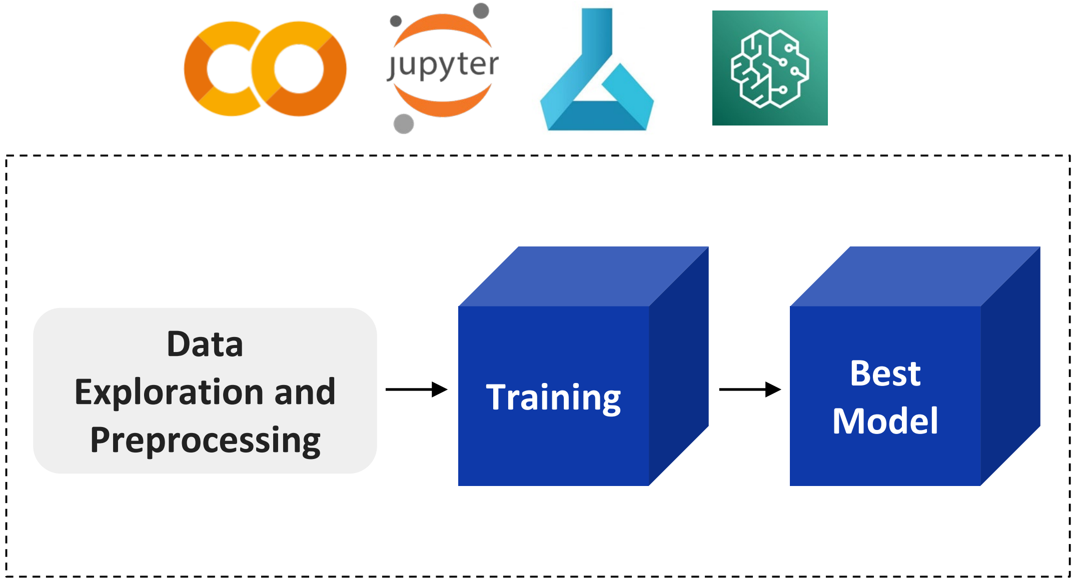
The Need for Serialization



Development Environment

- The best model is in the development environment and we need to make predictions for new data.
- What if we **shut down** the development environment?
- Need **to execute all data processing steps** and **build the best model again** from scratch.
- Loss of resources and time.

The Need for Serialization



Development Environment

- The best model is in the development environment and we need to make predictions for new data.
- What if we **shut down** the development environment?
- Need **to execute all data processing steps** and **build the best model again** from scratch.
- Loss of resources and time.
- We need a way to **"save" the best model** to save our progress.

This is where **serialization** comes in.

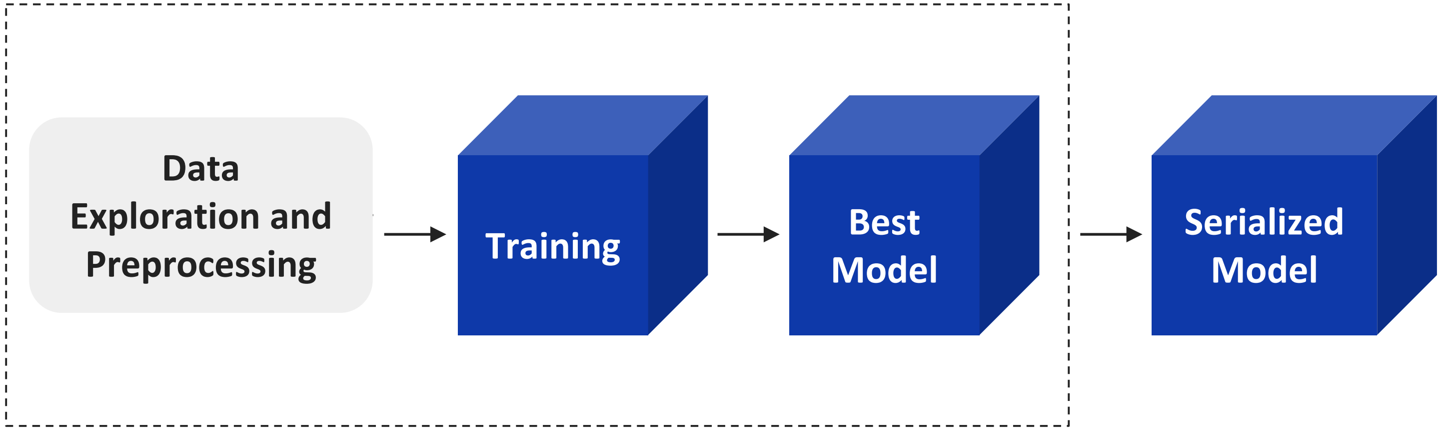
Video Break

Model Serialization

Serialization is the process of translating a data structure or object state into a format that can be stored or transmitted and reconstructed later.

Model Serialization

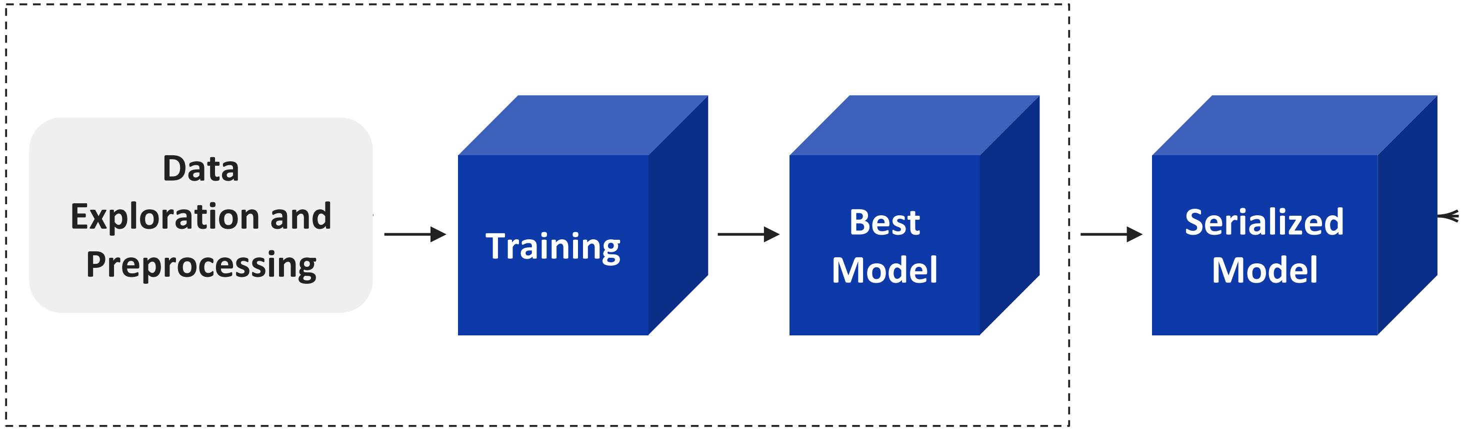
Serialization is the process of translating a data structure or object state into a format that can be stored or transmitted and reconstructed later.



Development Environment

Model Serialization

Serialization is the process of translating a data structure or object state into a format that can be stored or transmitted and reconstructed later.



Development Environment

- Can **load** it in **any environment**
- **Need not execute** the entire **code again**
- Can make **faster predictions**

Model Serialization Formats

There are **multiple formats** available for **serializing an ML model** to ensure **compatibility**, **efficiency**, and **ease of use**:

Model Serialization Formats

There are **multiple formats** available for **serializing an ML model** to ensure **compatibility**, **efficiency**, and **ease of use**:

Pickle	Saving and loading small models in python for quick and local use

Model Serialization Formats

There are **multiple formats** available for **serializing an ML model** to ensure **compatibility**, **efficiency**, and **ease of use**:

Pickle	Saving and loading small models in python for quick and local use
Joblib	Handling large NumPy-based models

Model Serialization Formats

There are **multiple formats** available for **serializing an ML model** to ensure **compatibility**, **efficiency**, and **ease of use**:

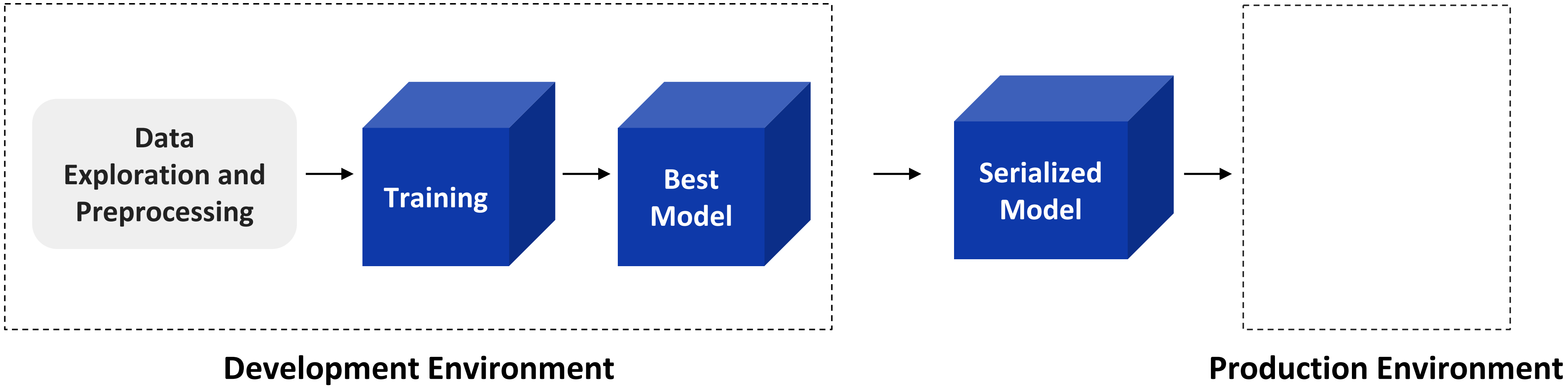
Format	Use
Pickle	Saving and loading small models in python for quick and local use
Joblib	Handling large NumPy-based models
ONNX	Deploying models across different frameworks or running them on edge devices

Model Serialization Formats

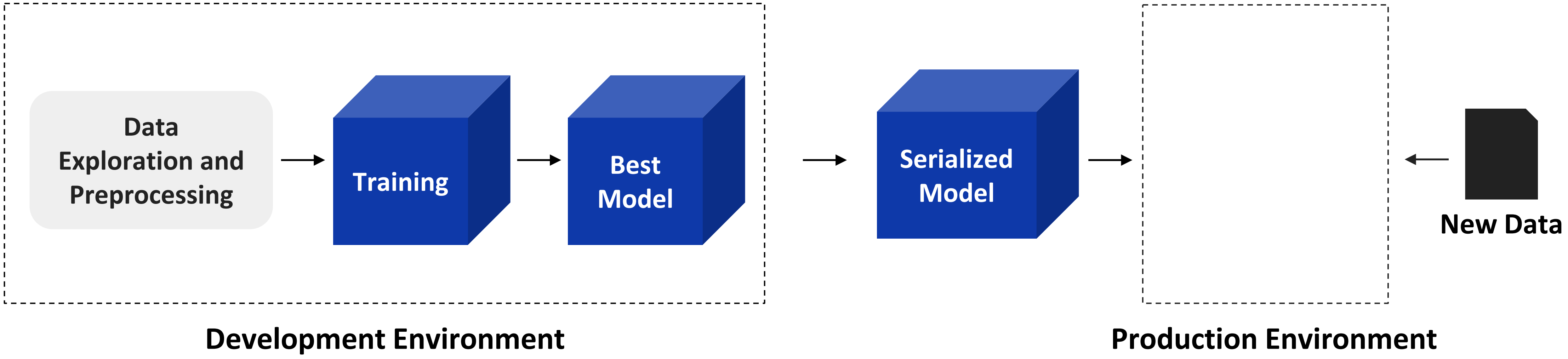
There are **multiple formats** available for **serializing an ML model** to ensure **compatibility**, **efficiency**, and **ease of use**:

Pickle	Saving and loading small models in python for quick and local use
Joblib	Handling large NumPy-based models
ONNX	Deploying models across different frameworks or running them on edge devices
TensorFlow SavedModel	Deploying deep learning models in production with TensorFlow serving or TensorFlow lite

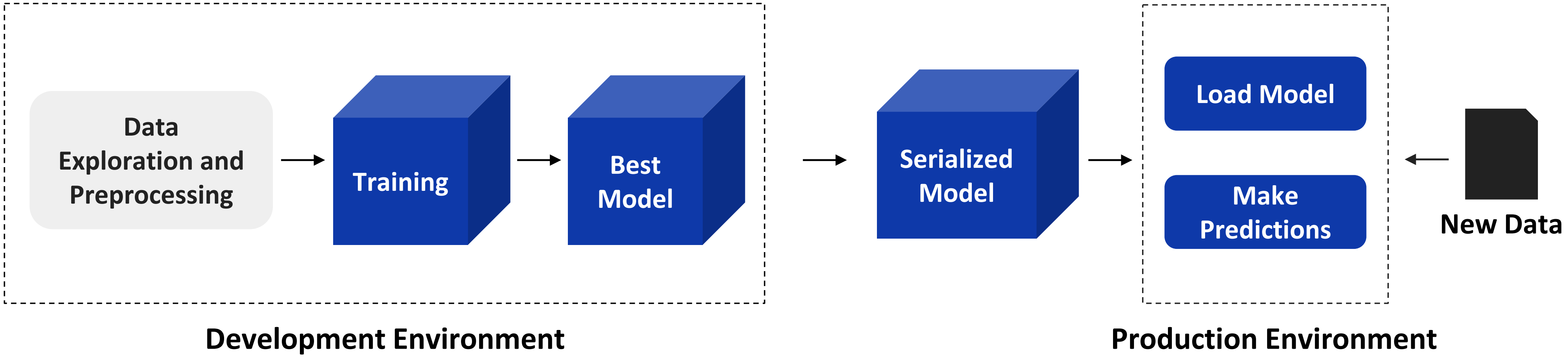
Serialization



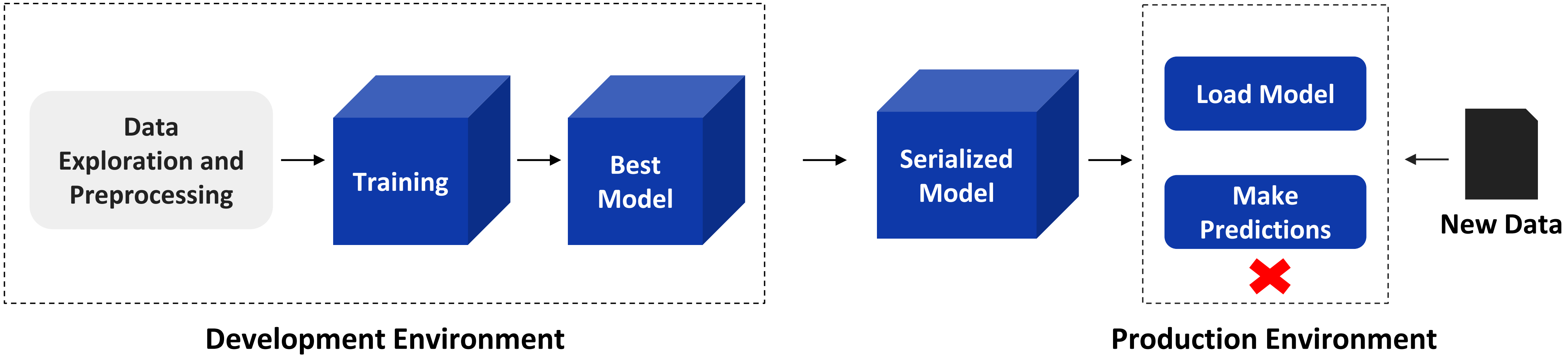
Serialization



Serialization

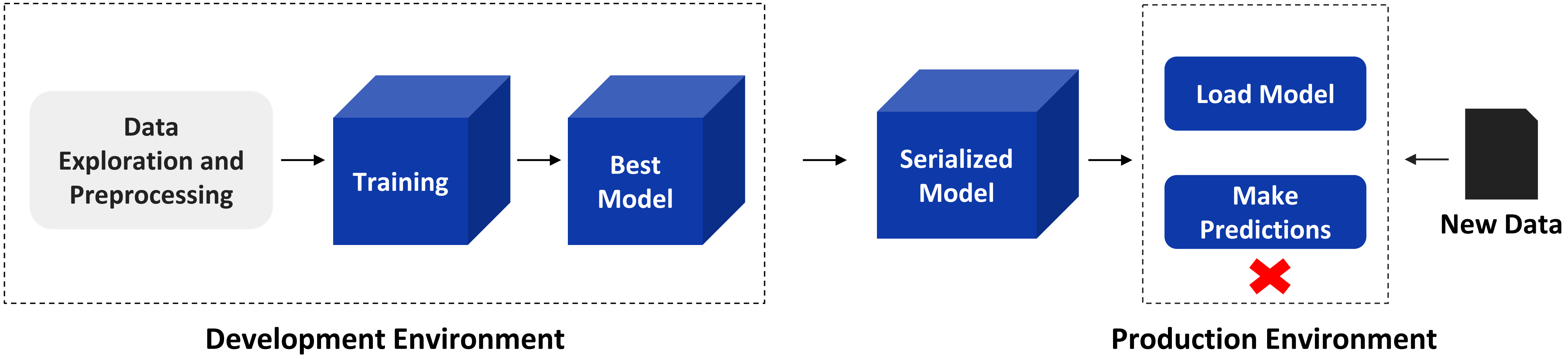


Serialization



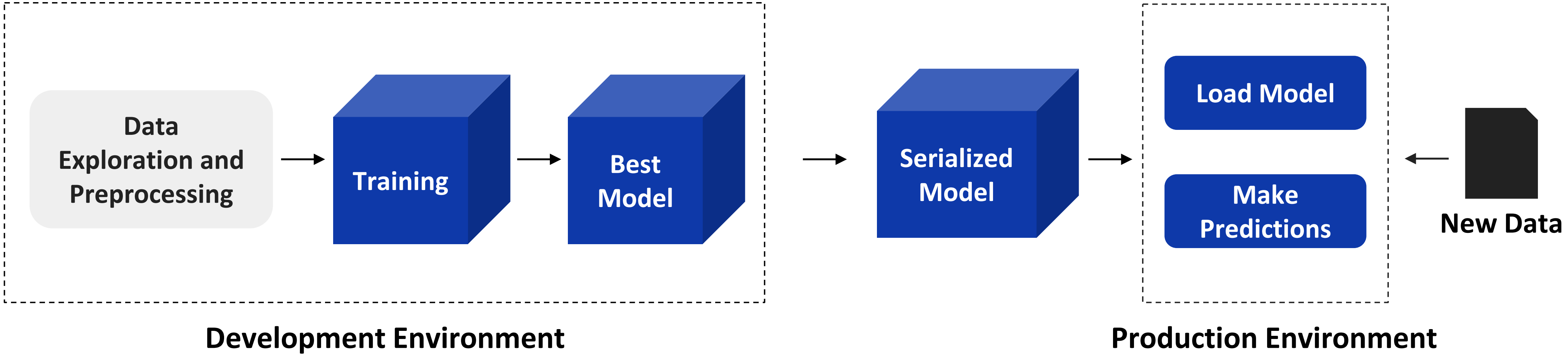
- Why did the predictions fail?

Serialization



- The **new data had additional genres** (say “Thriller” and “Horror”) beyond what was used to train the model (say “Action”, “Adventure”, “Sci-Fi”, “Romance”, and “Comedy”).

Serialization



- We need to **serialize the best model along with the data preprocessing steps** that were used to preprocess the data used to train the model.

Video Break

Addressing Notebook Sharing Challenges

A way to **share** the model

A way to **ensure consistent configurations**

Interface for communication to get fast predictions

Concurrent access for multiple users

1

2

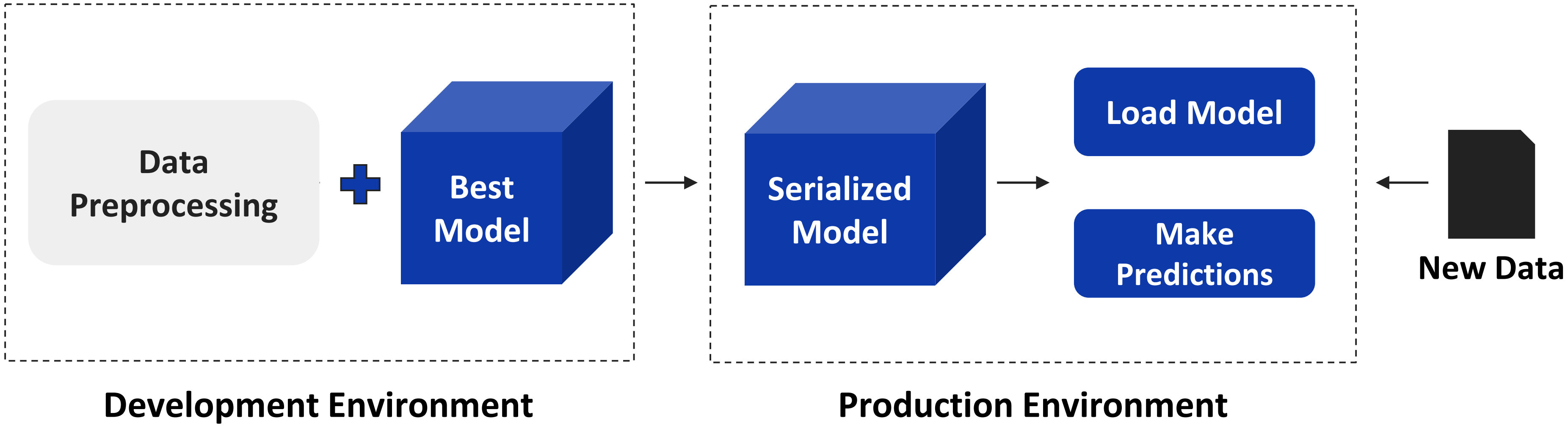
3

4

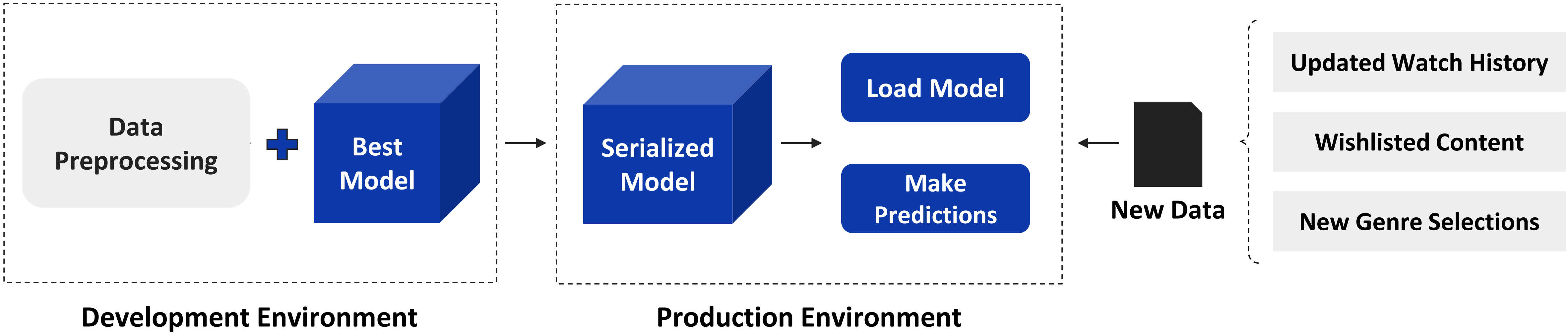
Serialized Model

- Next, let's discuss about the communication interface for faster predictions.

The Need for a Communication Interface

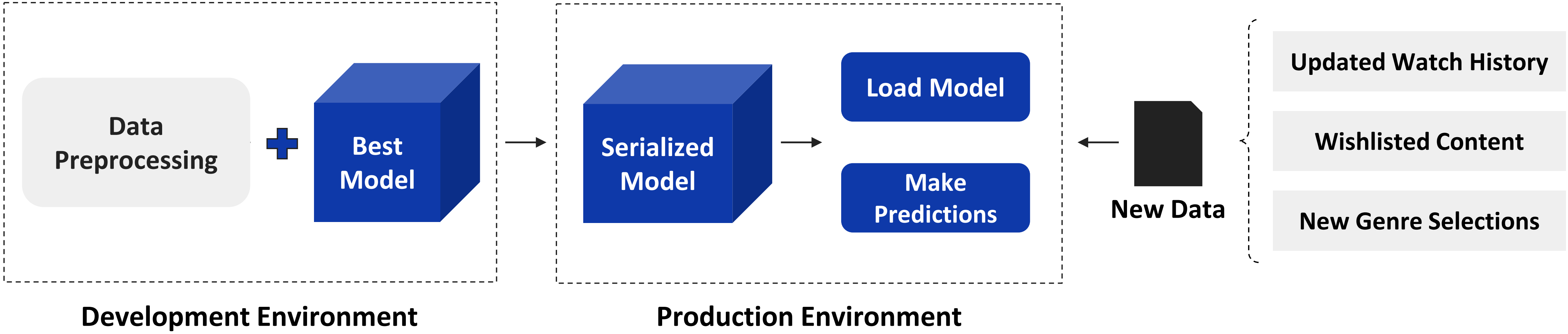


The Need for a Communication Interface



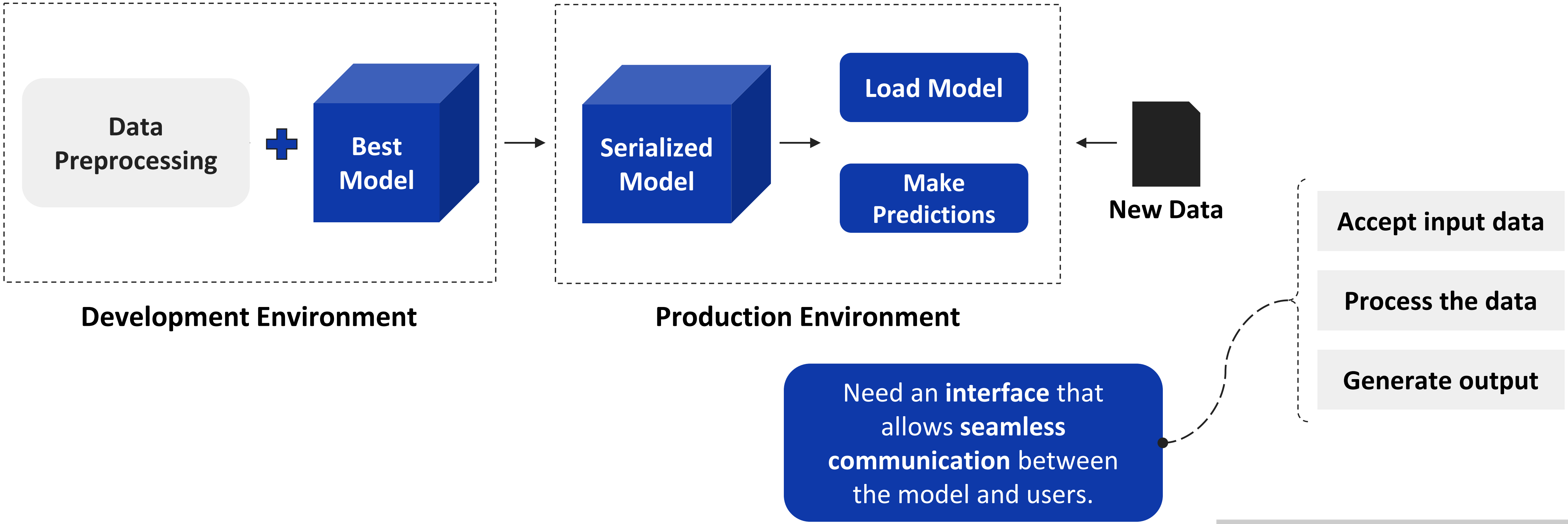
- How can the model receive the new user data and then generate recommendations accordingly?

The Need for a Communication Interface

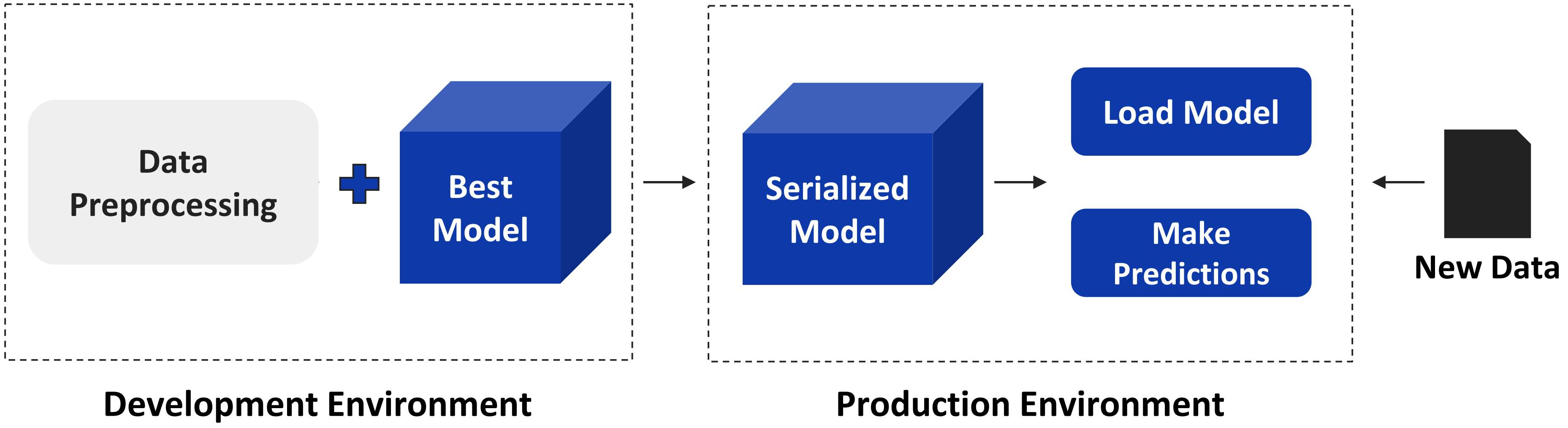


- How can the model receive the new user data and then generate recommendations accordingly?
- Important to note that we **DON'T** want the user **to access the model directly** as the raw output might not be useful to the end user.

The Need for a Communication Interface



The Need for a Communication Interface



- Accept input data
- Process the data
- Generate output

Need an **interface** that allows **seamless communication** between the model and users.

This is where an **API** comes in.

Application Programming Interface (API)

Application **P**rogramming **I**nterfaces, or **APIs**, are **mechanisms** that **enable two software components to communicate** with each other **using a set of definitions and protocols**.

Application Programming Interface (API)

Application **P**rogramming **I**nterfaces, or **APIs**, are **mechanisms** that **enable two software components to communicate** with each other **using a set of definitions and protocols**.

Netflix App

Application Programming Interface (API)

Application **P**rogramming **I**nterfaces, or **APIs**, are **mechanisms** that **enable two software components to communicate** with each other **using a set of definitions and protocols**.

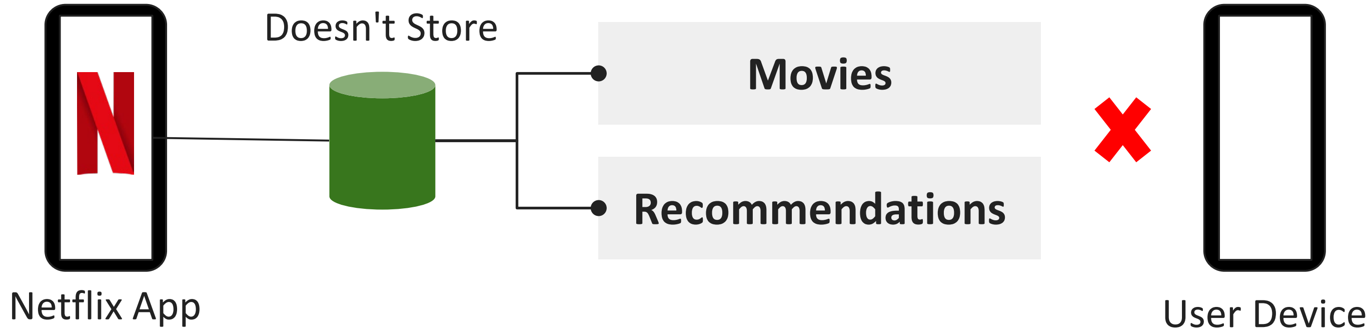
Netflix App

Movies

Recommendations

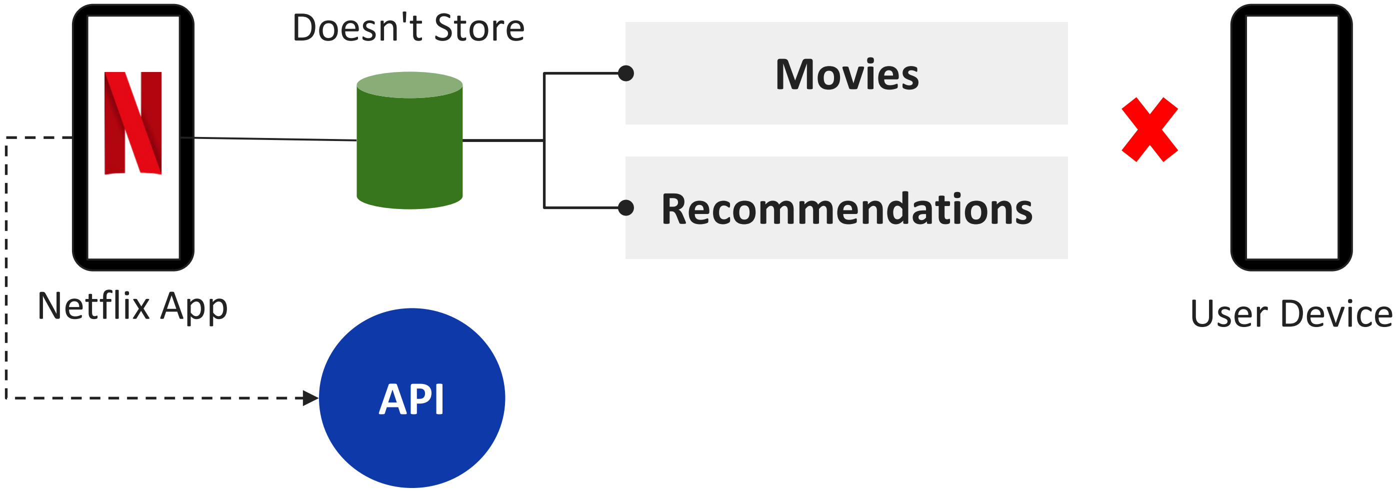
Application Programming Interface (API)

Application **P**rogramming **I**nterfaces, or **API**s, are **mechanisms** that **enable two software components to communicate** with each other **using a set of definitions and protocols**.



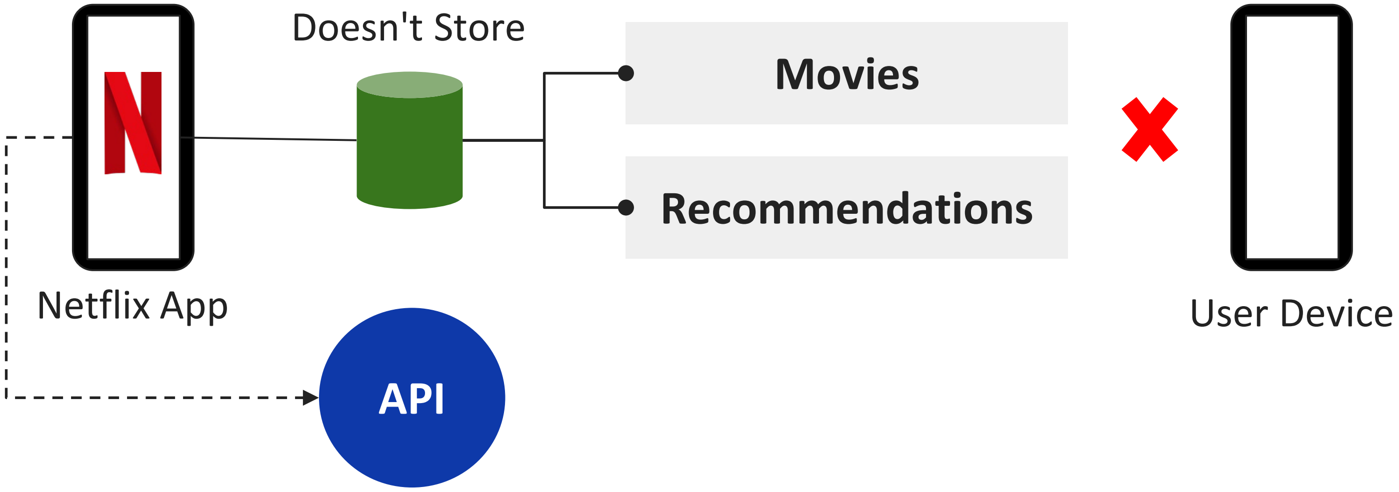
Application Programming Interface (API)

Application **P**rogramming **I**nterfaces, or **APIs**, are **mechanisms** that **enable two software components to communicate** with each other **using a set of definitions and protocols**.



Application Programming Interface (API)

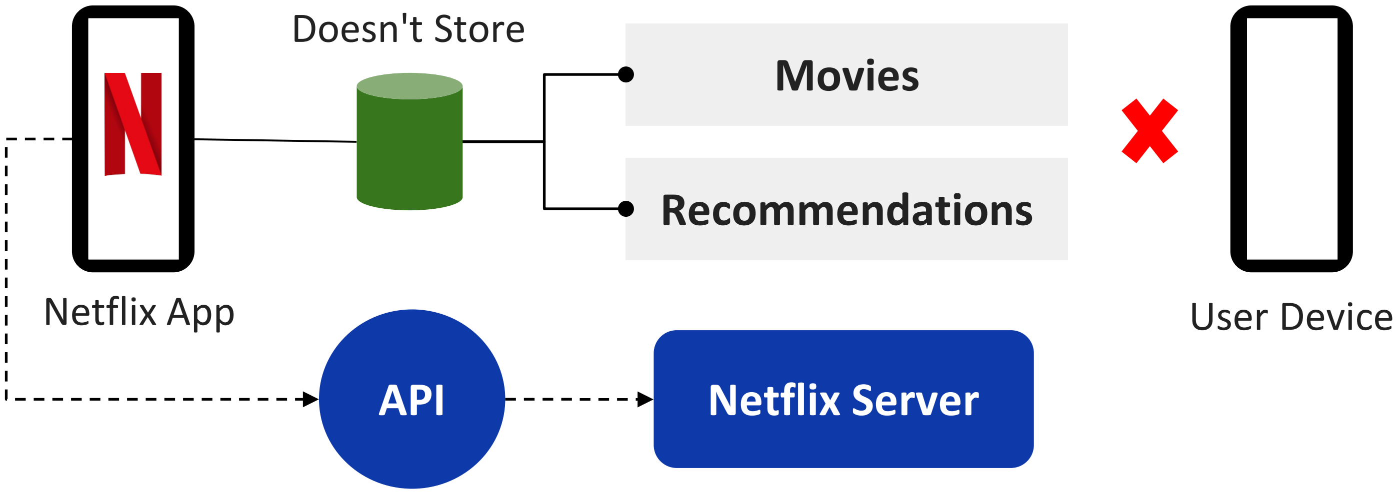
Application **P**rogramming **I**nterfaces, or **APIs**, are **mechanisms** that **enable two software components to communicate** with each other **using a set of definitions and protocols**.



- But then where is the data fetched from?

Server

A **server** is a **central computer** from which other computers get information.



HTTP Methods

- When a user wants to get predictions from a model via an API, they need to send a request.

An **API request**, or an **API call**, is a **message sent to a server** asking an API to **provide a service or information**.

HTTP Methods

- When a user wants to get predictions from a model via an API, they need to send a request.

An **API request**, or an **API call**, is a **message sent to a server** asking an API to **provide a service or information**.

- How does a user specify what kind of request it is?

HTTP Methods

- When a user wants to get predictions from a model via an API, they need to send a request.

An **API request**, or an **API call**, is a **message sent to a server** asking an API to **provide a service or information**.

- How does a user specify what kind of request it is?

Send Data	Profile information from new users.

HTTP Methods

- When a user wants to get predictions from a model via an API, they need to send a request.

An **API request**, or an **API call**, is a **message sent to a server** asking an API to **provide a service or information**.

- How does a user specify what kind of request it is?

Send Data	Profile information from new users.
Update Data	Modifications to user interests or preference.

HTTP Methods

- When a user wants to get predictions from a model via an API, they need to send a request.

An **API request**, or an **API call**, is a **message sent to a server** asking an API to **provide a service or information**.

- How does a user specify what kind of request it is?

Send Data	Profile information from new users.
Update Data	Modifications to user interests or preference.
Delete Data	Clearing user watch history.

HTTP Methods

- This is where **HTTP methods** come in.
- They **define** the **type of interaction** we want with the API.

HTTP Methods

- This is where **HTTP Methods** come in.
- They **define** the **type of interaction** we want with the API.

GET	**Retrieve data from an API**	Get a movie recommendation.
POST	**Send data to the API**	Send current watchlist to the model.
PUT	**Update existing data**	Add new movie to the watchlist.
DELETE	**Remove data**	Clear watch history.

- Note that all these operations **DON'T** require an ML model.

HTTP Methods

- A user made an API request, but didn't get the correct output.

HTTP Methods

- A user made an API request, but didn't get the correct output.
- We need to know the reason behind the incorrect output (Missing data, model loading issue, server issue, or something else).

HTTP Methods

- A user made an API request, but didn’t get the correct output.
- We need to know the reason behind the incorrect output (Missing data, model loading issue, server issue, or something else).
- Once the request is processed, it is important for an API to respond with a status indicating whether it succeeded or failed.

HTTP Methods

- A user made an API request, but didn't get the correct output.
- We need to know the reason behind the incorrect output (Missing data, model loading issue, server issue, or something else).
- Once the request is processed, it is important for an API to respond with a status indicating whether it succeeded or failed.
- **HTTP status codes** provide this information, ensuring smooth communication between users and the API.

HTTP Status Codes

Status Code	Status Message	Description
200	Ok	Request was successful.

HTTP Status Codes

Status Code	Status Message	Description
200	Ok	Request was successful.
400	Bad Request	The request was invalid.

HTTP Status Codes

Status Code	Status Message	Description
200	Ok	Request was successful.
400	Bad Request	The request was invalid.
401	Unauthorised	Authentication is required.

HTTP Status Codes

Status Code	Status Message	Description
200	Ok	Request was successful.
400	Bad Request	The request was invalid.
401	Unauthorised	Authentication is required.
403	Forbidden	Access is not allowed.

HTTP Status Codes

Status Code	Status Message	Description
200	Ok	Request was successful.
400	Bad Request	The request was invalid.
401	Unauthorised	Authentication is required.
403	Forbidden	Access is not allowed.
404	Not Found	Requested resource does not exist.

HTTP Status Codes

Status Code	Status Message	Description
200	Ok	Request was successful.
400	Bad Request	The request was invalid.
401	Unauthorised	Authentication is required.
403	Forbidden	Access is not allowed.
404	Not Found	Requested resource does not exist.
500	Internal Server Error	The server encountered an error.

Video Break

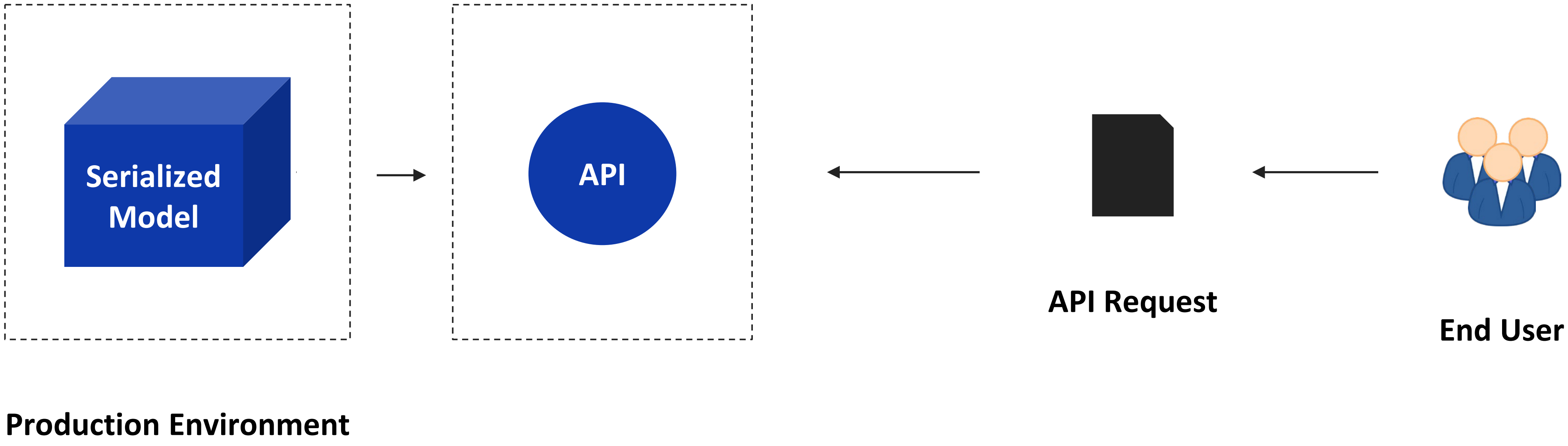
Addressing Notebook Sharing Challenges

1	2	3	4
A way to **share** the model	A way to **ensure consistent configurations**	**Interface for communication** to get fast predictions	**Concurrent access** for multiple users
Serialized Model		**API**	

- Let’s now see how can we solve the challenge of “concurrent access for multiple users”.

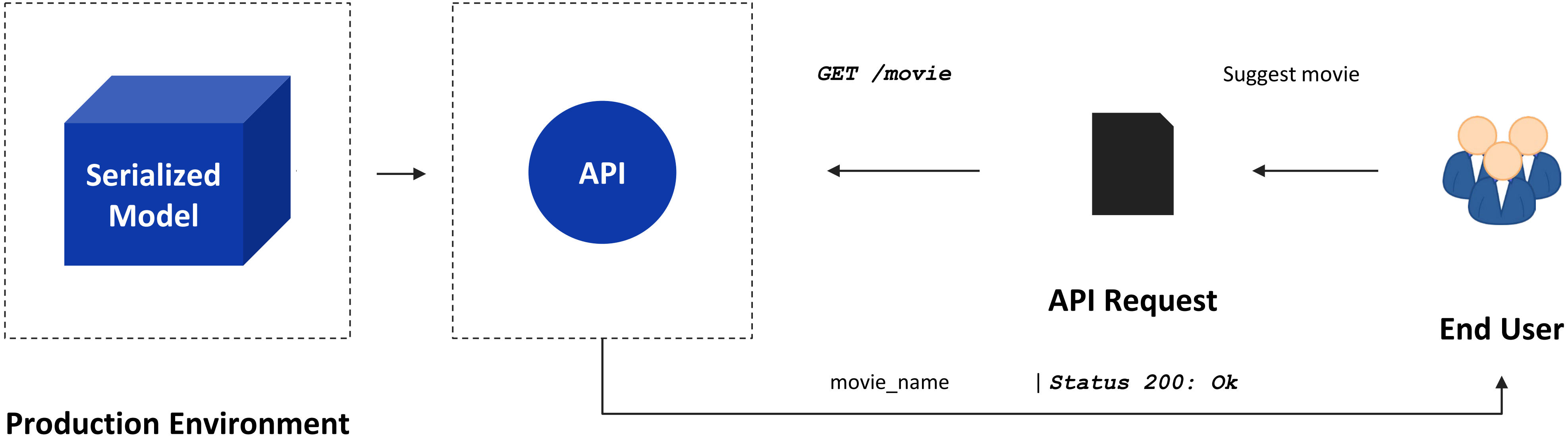
The Need for Endpoints

- A user made an API request to get a movie recommendation.



The Need for Endpoints

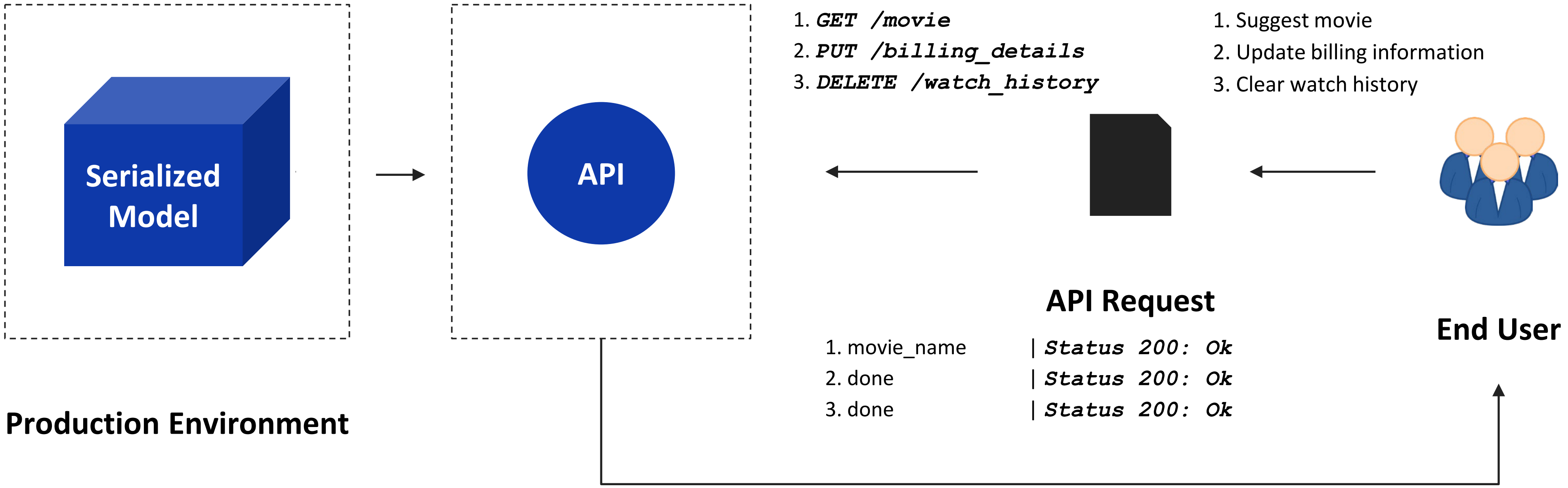
- A user made an API request to get a movie recommendation.



- The **instruction** to the API and the **response** from the API are **technical** in nature.

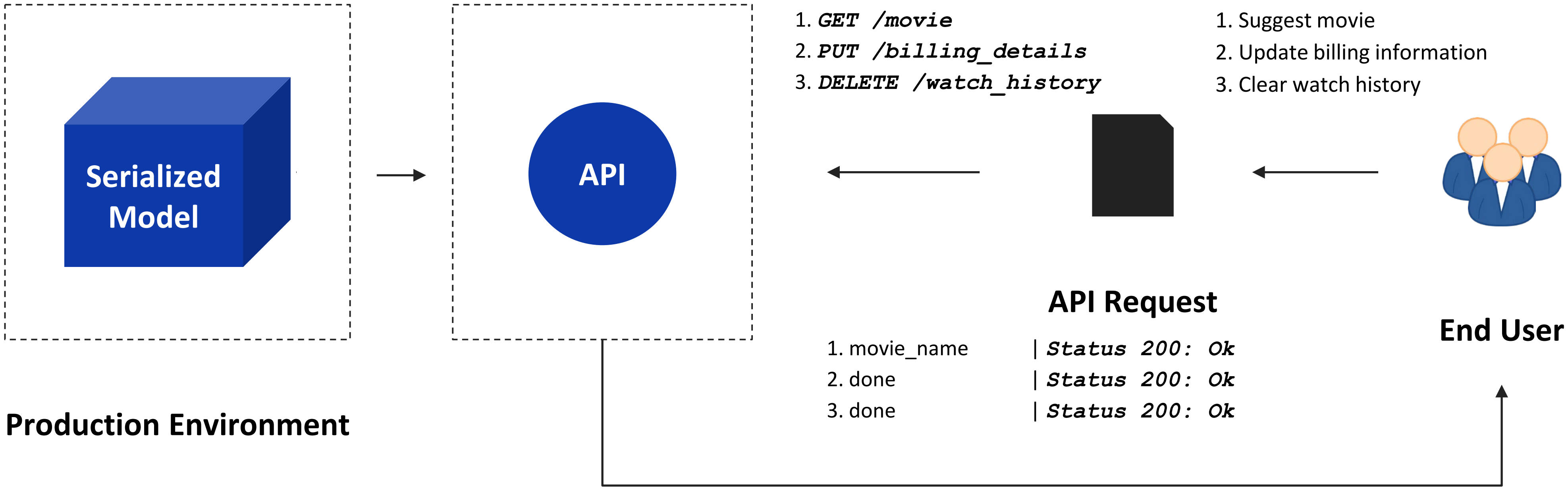
The Need for Endpoints

- A user can make multiple requests for multiple tasks.



The Need for Endpoints

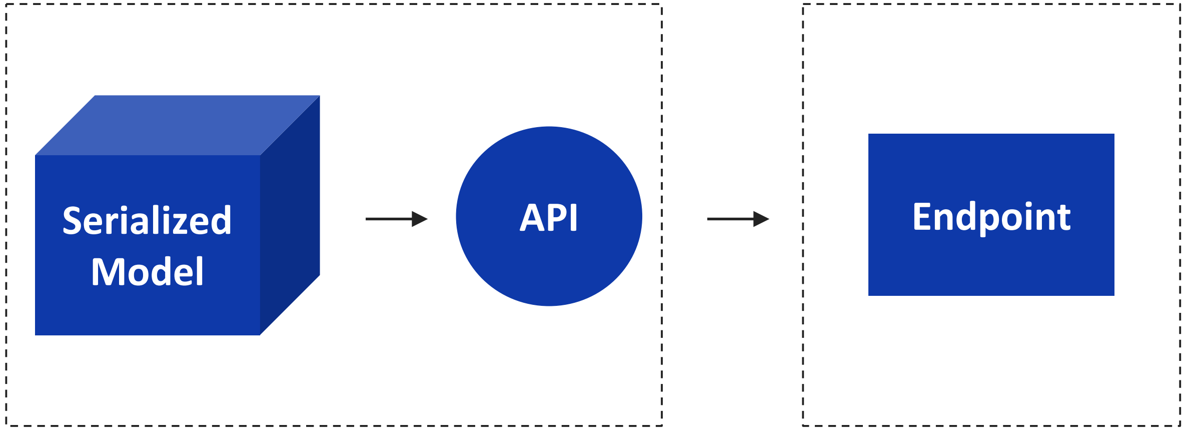
- A user can make multiple requests for multiple tasks.



- It’s **not feasible** for a user to **remember** all **the instructions and status codes.**

Endpoints

- We can **create an endpoint from an API** to make things **simpler for the end user**.

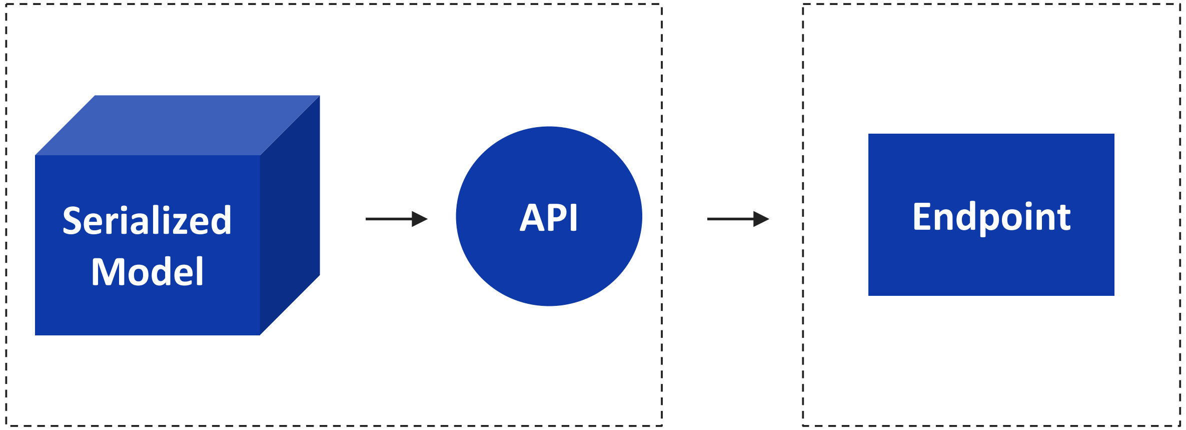


Production Environment

Endpoints

- We can **create an endpoint from an API** to make things **simpler for the end user**.

An **API endpoint** is a **digital location** where an **API receives API requests**, for resources on its server.



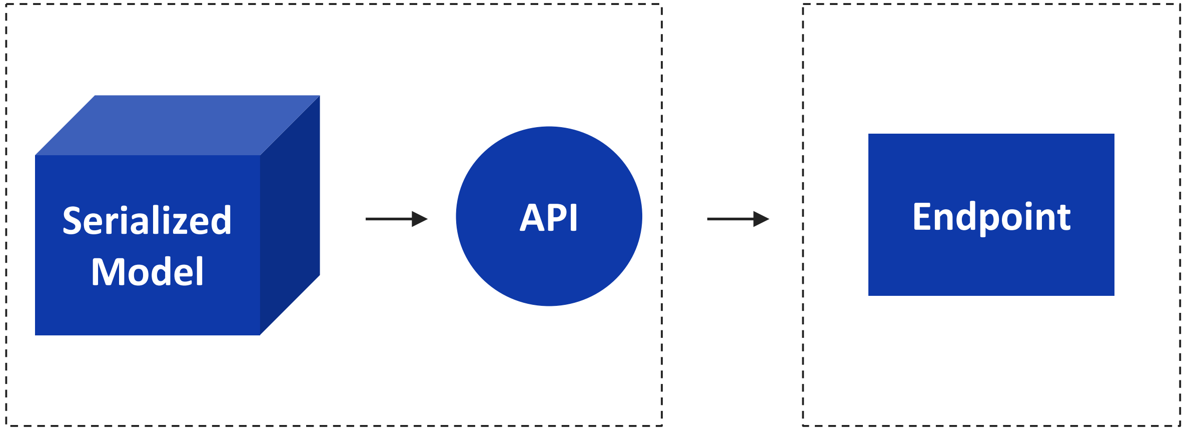
Production Environment

Endpoints

- We can **create an endpoint from an API** to make things **simpler for the end user**.

An **API endpoint** is a **digital location** where an **API receives API requests**, for resources on its server.

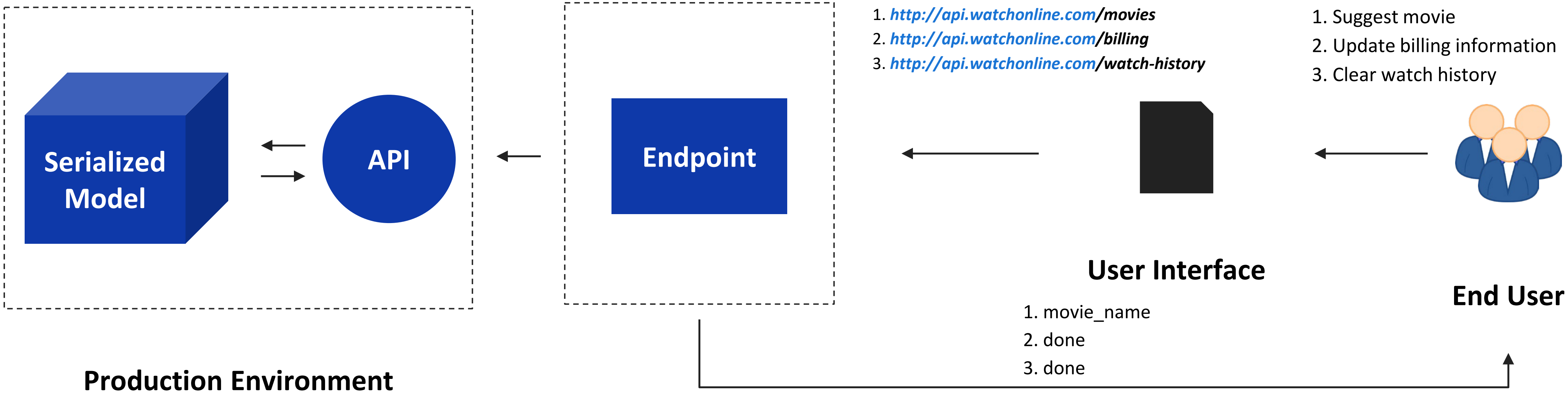
- They are most often in the form of URLs (**U**niform **R**esource **L**ocators).



Production Environment

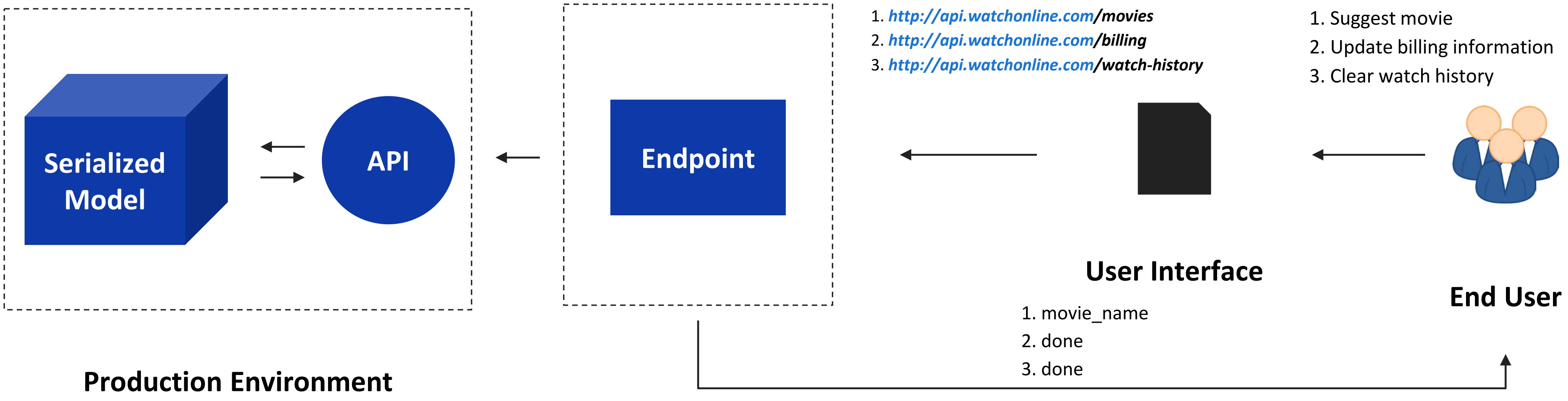
Endpoints

- Each functionality (API request) is performed via an endpoint (URL), which is generally a user interface.



Endpoints

- Each functionality (API request) is performed via an endpoint (URL), which is generally a user interface.



- Note that **a part of the URL remains the same**, while **other parts change as per the user request.**

Hosting an Endpoint

- How do we create an endpoint that allows multiple users to interact with the model?

Hosting an Endpoint

- How do we create an endpoint that allows multiple users to interact with the model?
- We need to host the endpoint.

Hosting is the **ability** to make applications and websites available on the internet.

Hosting an Endpoint

- How do we create an endpoint that allows multiple users to interact with the model?
- We need to host the endpoint.

Hosting is the **ability** to make applications and websites available on the internet.

- There are multiple ways to host an endpoint, but the two most common ways are the following:

Local Machine	**Self-hosting the model on a local server, accessible only within a restricted environment.**
Cloud Platform	**Hosting the model on cloud platforms (like AWS, Azure, or GCP) making it accessible globally.**

Video Break

Addressing Notebook Sharing Challenges

A way to **share** the model

A way to **ensure consistent configurations**

Interface for communication to get fast predictions

Concurrent access for multiple users

1 **Serialized Model**

2

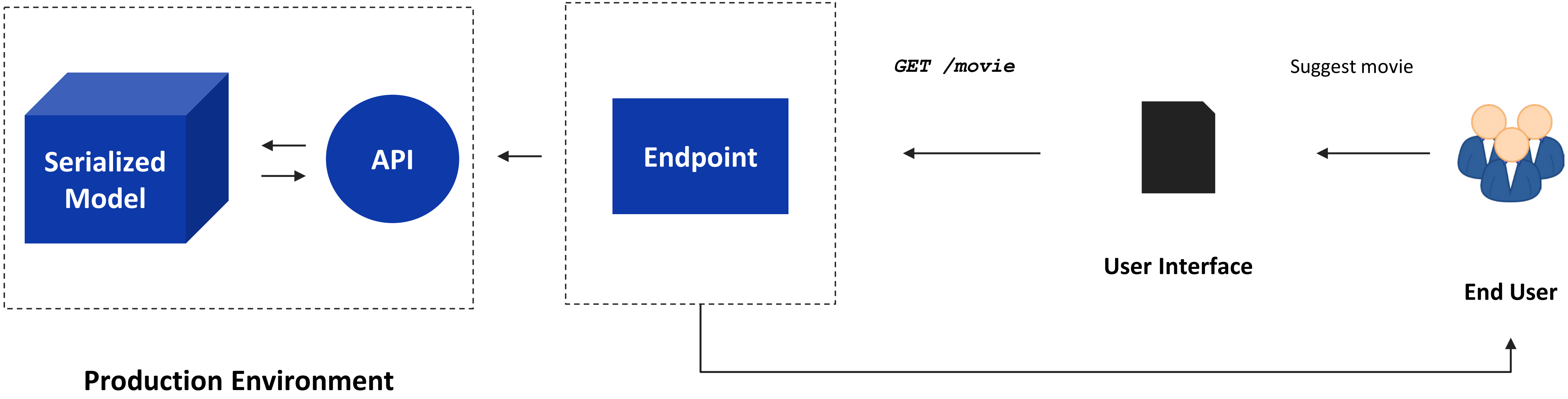
3 **API**

4 **Hosted Endpoint**

- Now, let's solve the challenge of consistent configurations (environments).

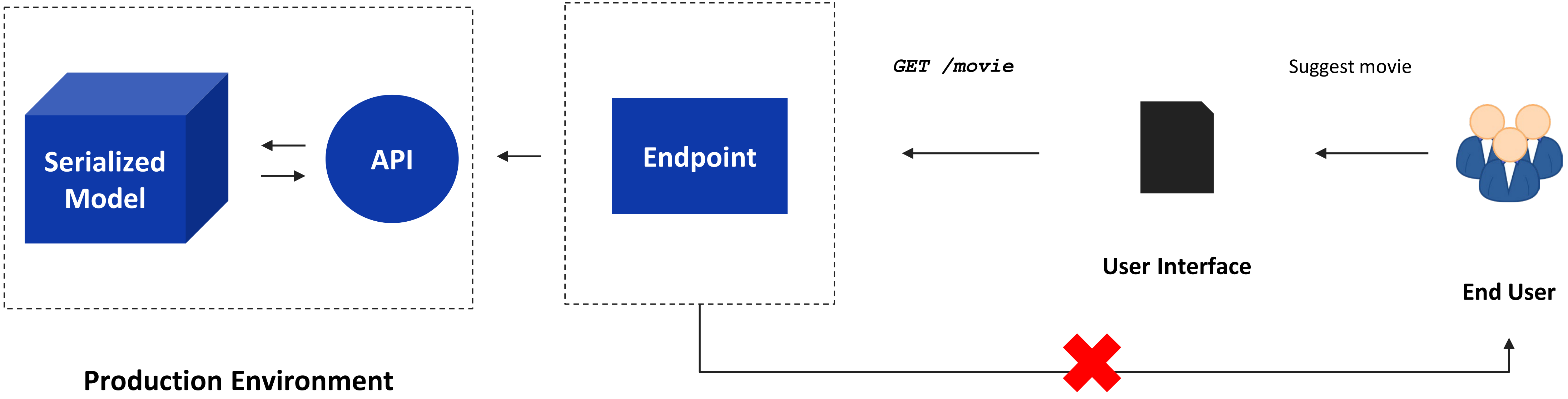
Role of Dependencies

- A user tried to get a movie recommendation from the API endpoint.



Role of Dependencies

- A user tried to get a movie recommendation from the API endpoint.



- Let's see why the user failed to get a response.

Role of Dependencies

Has the serialized model and the API

Production Environment

✘

Deployment failed

▼

Role of Dependencies

Has the serialized model and the API

Production Environment

Deployment failed

Error: No module named sklearn

Role of Dependencies

Has the serialized model and the API

Production Environment

Deployment failed

▼

Error: No module named sklearn

▼

pip install sklearn

Developer tried to install the missing library in the production environment

Role of Dependencies

Has the serialized model and the API

Production Environment

Deployment failed

Error: No module named sklearn

pip install sklearn

Developer tried to install the missing library in the production environment

Error: No module named pandas

pip install pandas

Developer again tried to install the missing library in the production environment

Role of Dependencies

Has the serialized model and the API

Production Environment

Deployment failed

Error: No module named sklearn

```
pip install sklearn
```

Developer tried to install the missing library in the production environment

Error: No module named pandas

```
pip install pandas
```

Developer again tried to install the missing library in the production environment

Error: could not find the required version numpy==1.9.3

```
pip install numpy==1.9.3
```

Developer was frustrated, but tried to install required version of the library in the production environment

Role of Dependencies

Has the serialized model and the API

Production Environment

Deployment failed

Error: No module named sklearn

`pip install sklearn`

Developer tried to install the missing library in the production environment

Error: No module named pandas

`pip install pandas`

Developer again tried to install the missing library in the production environment

Error: could not find the required version numpy==1.9.3

`pip install numpy==1.9.3`

Developer was frustrated, but tried to install required version of the library in the production environment

- This is inefficient way to handle dependencies every time there's a failure.

Handling Dependencies

- What if we keep all dependencies at a single place and take care of them at one go?

Handling Dependencies

- What if we keep all dependencies at a single place and take care of them at one go?

We can **create a dependencies file** that'll act as a **blueprint for recreating** the exact **environment** where the model was trained and tested (the development environment).

Handling Dependencies

- What if we keep all dependencies at a single place and take care of them at one go?

We can **create a dependencies file** that'll act as a **blueprint for recreating** the exact **environment** where the model was trained and tested (the development environment).

- In general, dependencies files are specified as a text file **(requirements.txt)**.

requirements.txt

```
sklearn==1.6.1
pandas==2.2.3
numpy==2.2.4
..
..
```

Handling Dependencies

- What if we keep all dependencies at a single place and take care of them at one go?

We can **create a dependencies file** that'll act as a **blueprint for recreating** the exact **environment** where the model was trained and tested (the development environment).

- In general, dependencies files are specified as a text file **(requirements.txt)**.
- This creates a stable and reproducible environment, allowing the model to be deployed seamlessly and function as expected in production environment.

requirements.txt

```
sklearn==1.6.1
pandas==2.2.3
numpy==2.2.4
..
..
```

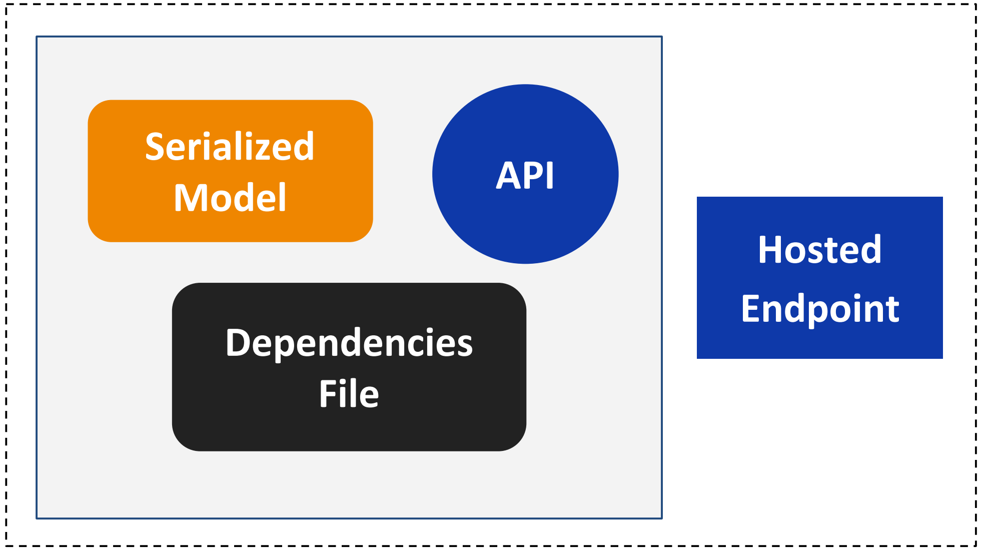
Video Break

Addressing Notebook Sharing Challenges

A way to **share** the model	A way to **ensure consistent configurations**	**Interface for communication** to get fast predictions	**Concurrent access** for multiple users
1	2	3	4
Serialized Model	**Dependencies File**	**API**	**Hosted Endpoint**

- Now that we have resolved all the challenges, we can deploy the model to make it accessible for users who want to make inferences.

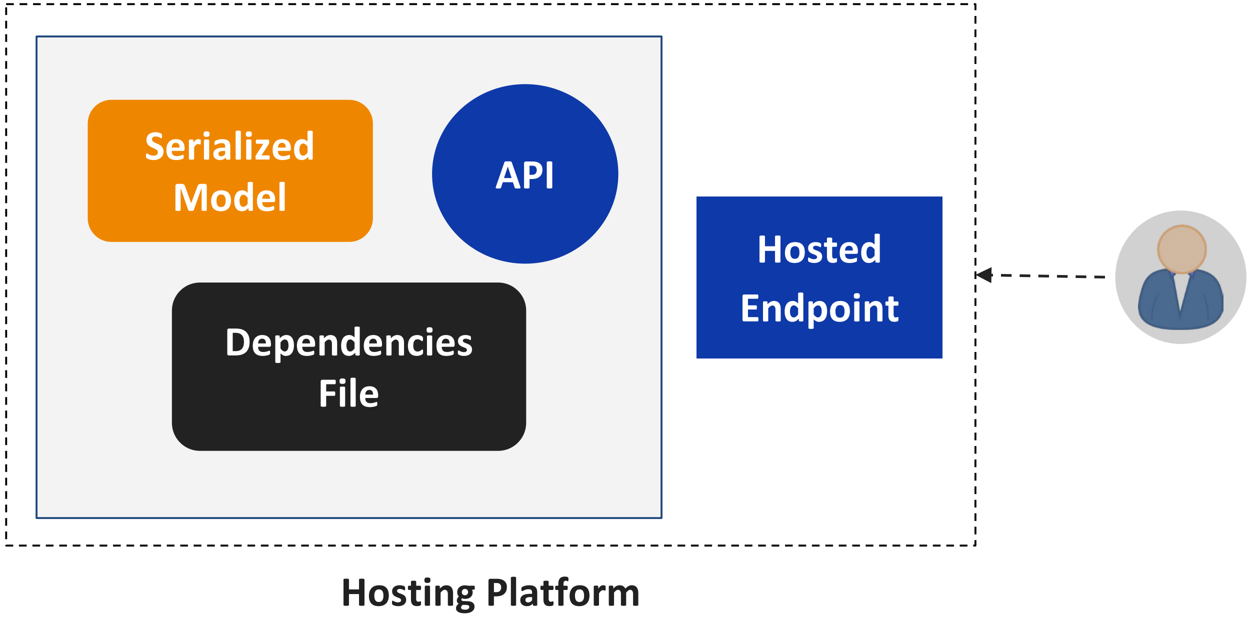
Model Deployment



Hosting Platform

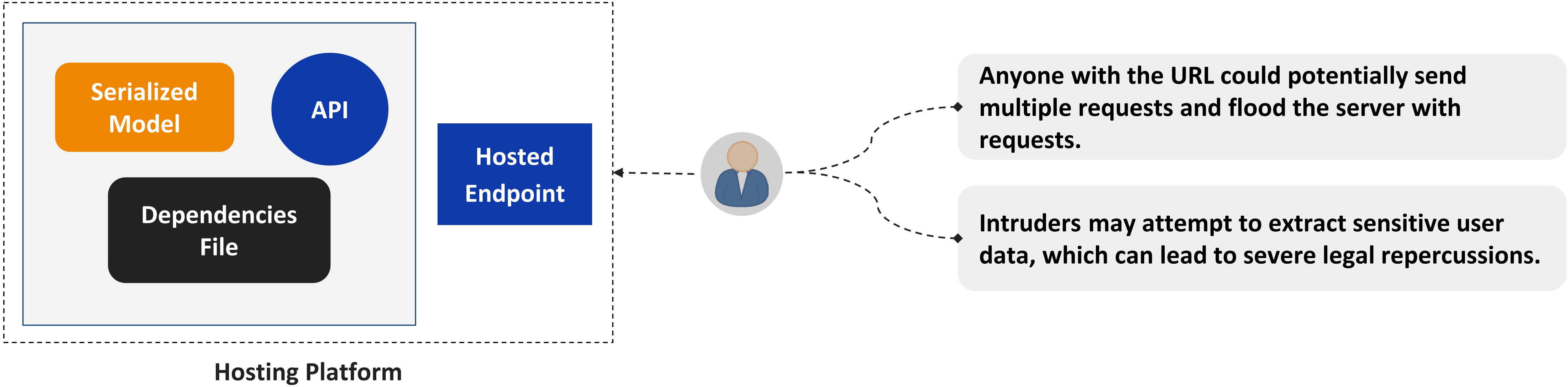
The Need for Security

- Consider the case where a user creates multiple fake accounts and starts sending too many requests.



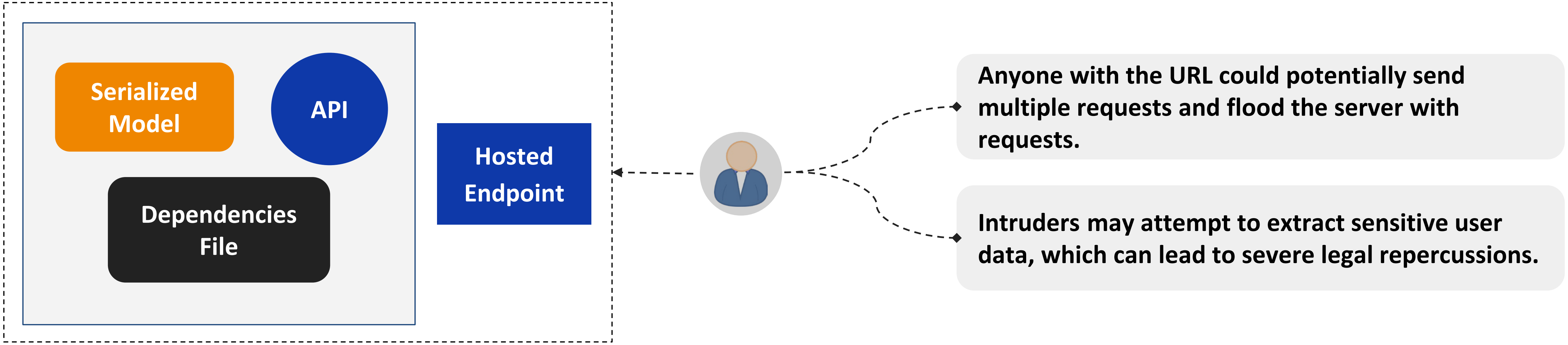
The Need for Security

- Consider the case where a user creates multiple fake accounts and starts sending too many requests.



The Need for Security

- Consider the case where a user creates multiple fake accounts and starts sending too many requests.



Hosting Platform

- How do we **ensure only trusted users interact with the service** (recommendation model)?

Access Keys

- This is where **access keys** come in.

An **access key** is a **unique identifier used to authenticate and authorize** a user or application to **access a secure resource.**

Access Keys

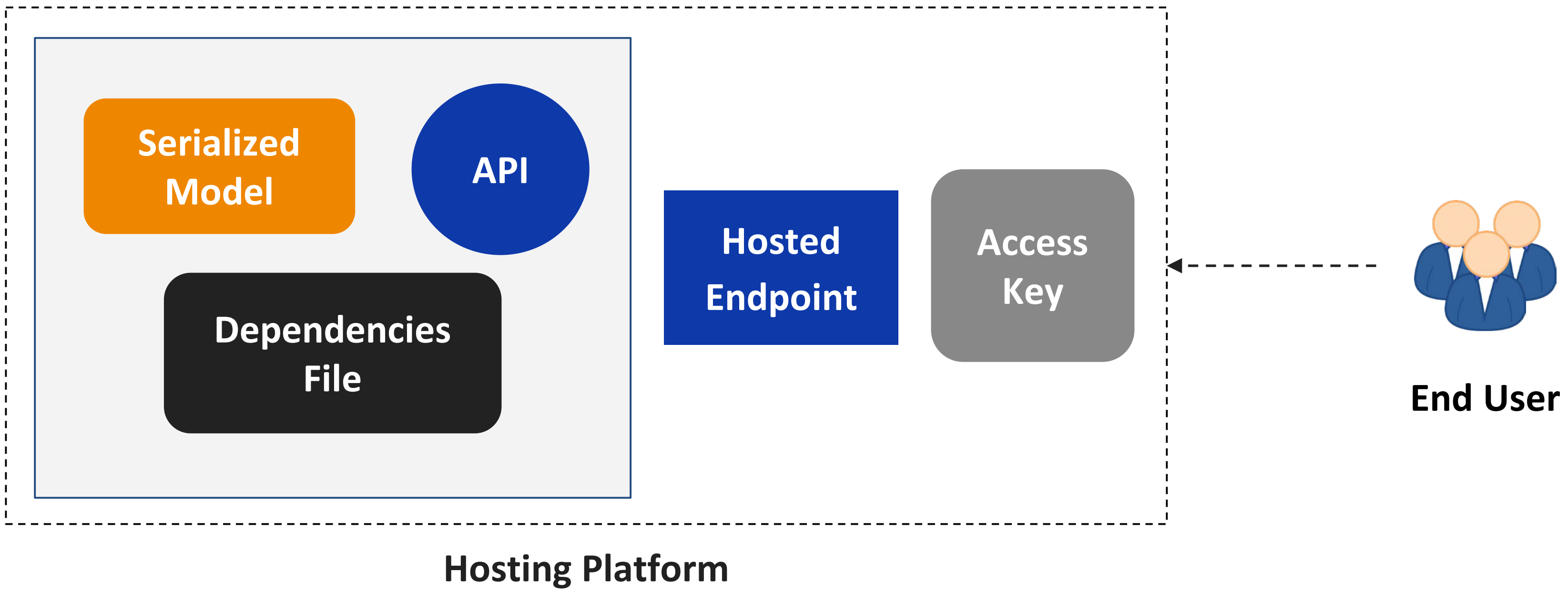
- This is where **access keys** come in.

An **access key** is a **unique identifier used to authenticate and authorize** a user or application to **access a secure resource.**

- **Access keys prevent unauthorized** access to secure resources (like data, models, and APIs).
- Only authorized users can use the resources and functionality associated with a specific account.

Secure Model Deployment

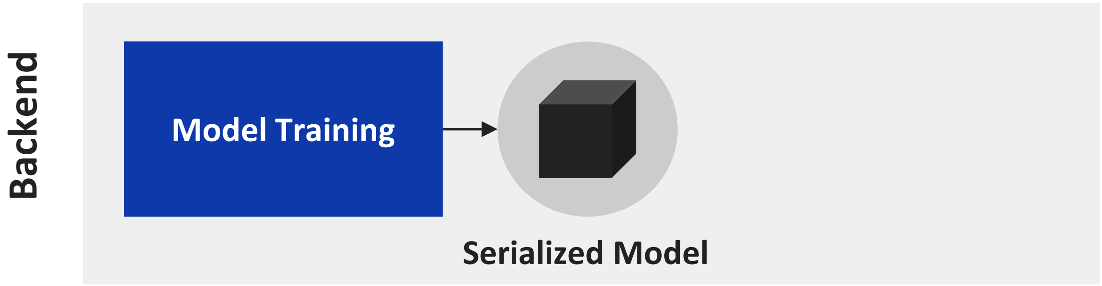
- Only authorized users will be able to securely access the public URL (endpoint).



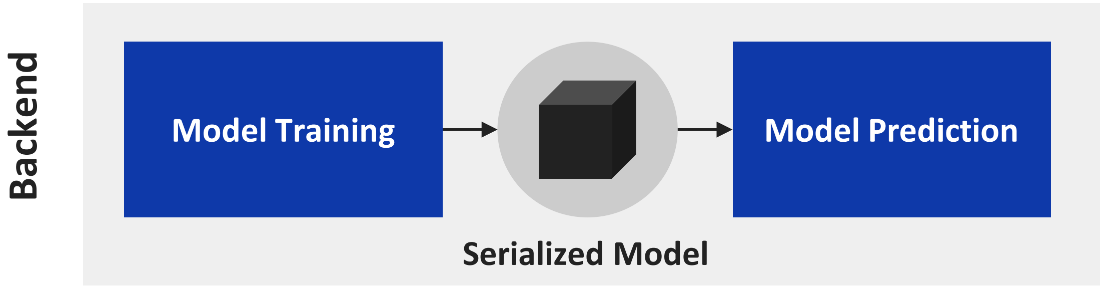
Video Break

Architecture of Model Deployment

Architecture of Model Deployment

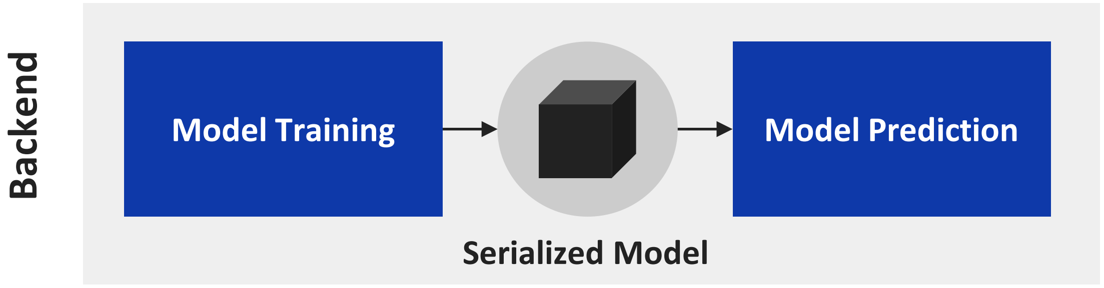


Architecture of Model Deployment

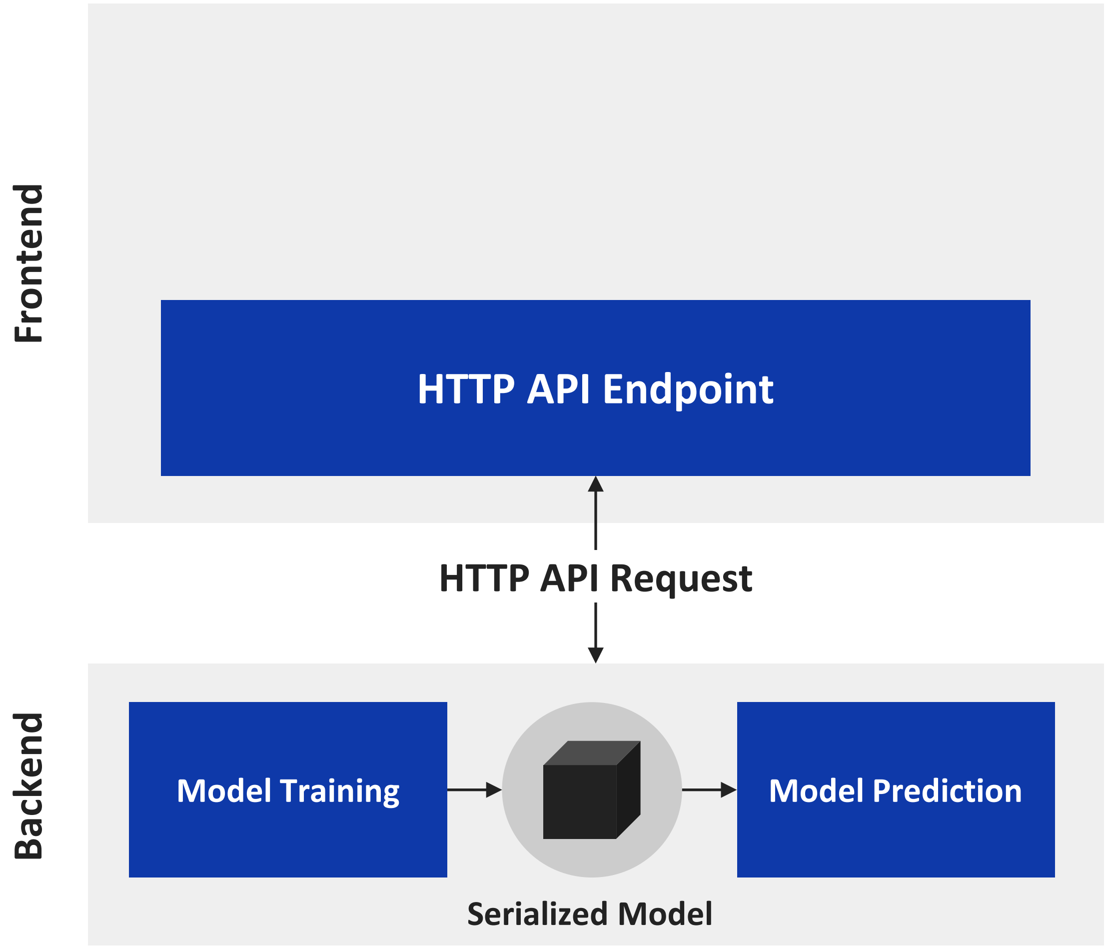


Architecture of Model Deployment

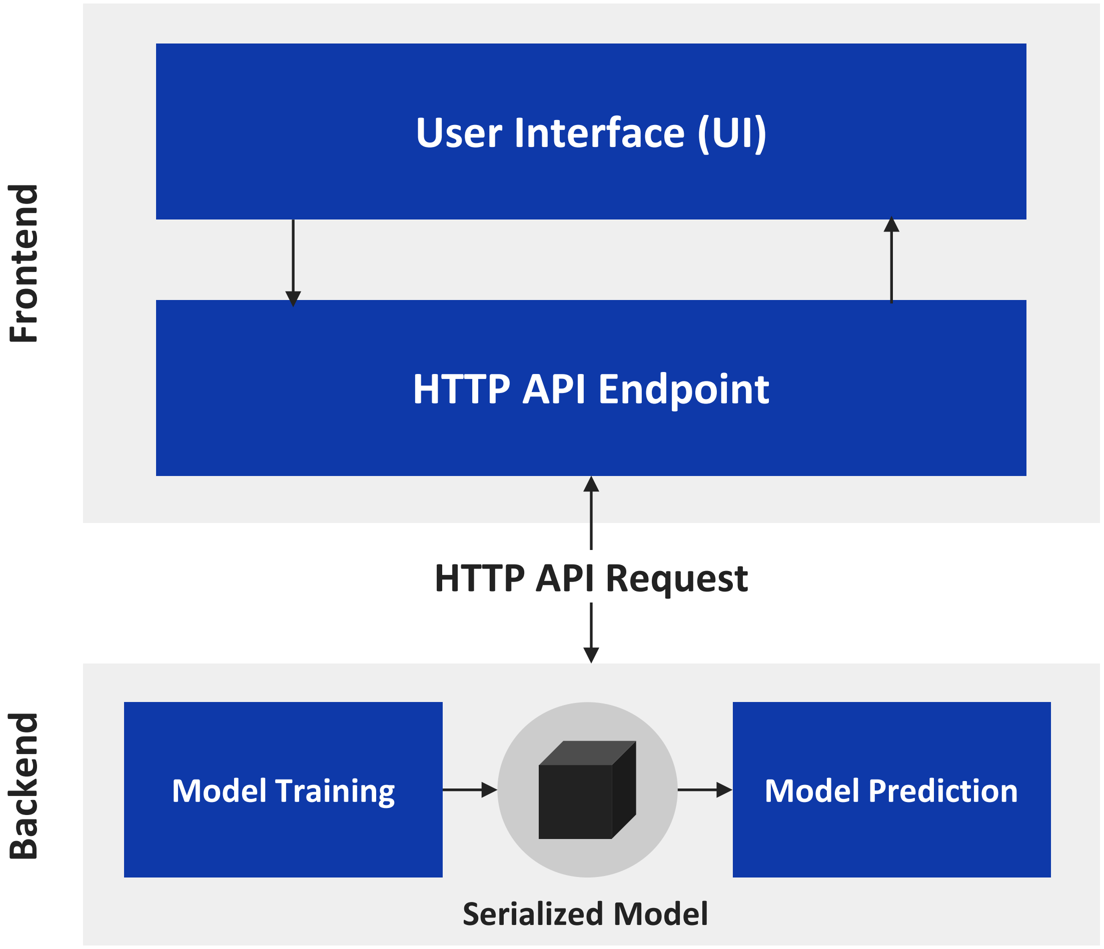
HTTP API Request



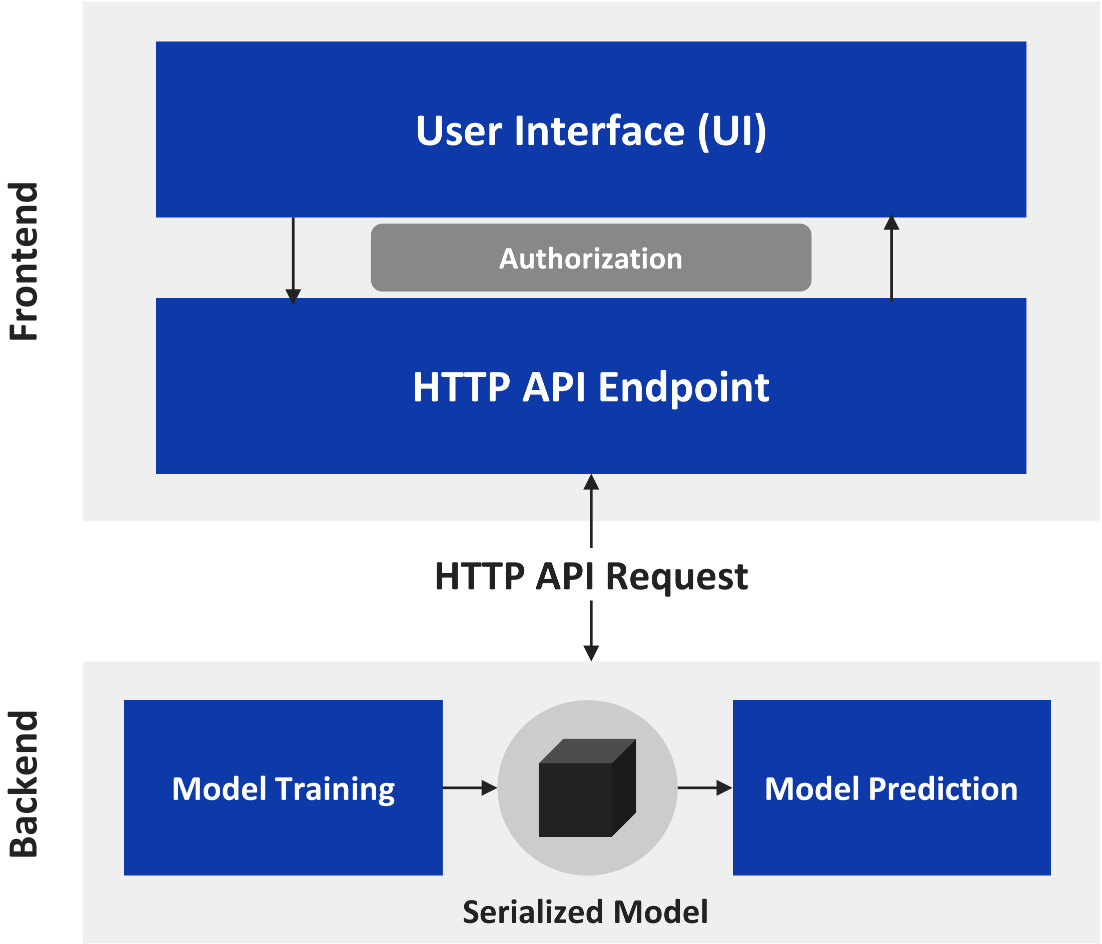
Architecture of Model Deployment



Architecture of Model Deployment



Architecture of Model Deployment



Video Break

Quick Recap

- **Problems Solved using ML**
- **Model Building Process**
- **Need for Deployment**

Scope of ML

Quick Recap

Scope of ML
- Problems Solved using ML
- Model Building Process
- Need for Deployment

The Challenges
- Sharing and Consistency
- Speed and Scalability

Quick Recap

Scope of ML
- Problems Solved using ML
- Model Building Process
- Need for Deployment

The Challenges
- Sharing and Consistency
- Speed and Scalability

Model Deployment
- Definition
- What It Solves For

Quick Recap

- **Scope of ML**
  - Problems Solved using ML
  - Model Building Process
  - Need for Deployment
- **The Challenges**
  - Sharing and Consistency
  - Speed and Scalability
- **Model Deployment**
  - Definition
  - What It Solves For
- **Serialization**
  - Definition
  - Formats

Quick Recap

Scope of ML
- Problems Solved using ML
- Model Building Process
- Need for Deployment

The Challenges
- Sharing and Consistency
- Speed and Scalability

Model Deployment
- Definition
- What It Solves For

Serialization
- Definition
- Formats

API
- Definition
- HTTP Methods and Status Codes

Quick Recap

- **Scope of ML**
  - Problems Solved using ML
  - Model Building Process
  - Need for Deployment
- **The Challenges**
  - Sharing and Consistency
  - Speed and Scalability
- **Model Deployment**
  - Definition
  - What It Solves For
- **Serialization**
  - Definition
  - Formats
- **API**
  - Definition
  - HTTP Methods and Status Codes
- **Endpoints**
  - Definition
  - Hosting

Quick Recap

Scope of ML
- Problems Solved using ML
- Model Building Process
- Need for Deployment

The Challenges
- Sharing and Consistency
- Speed and Scalability

Model Deployment
- Definition
- What It Solves For

Serialization
- Definition
- Formats

API
- Definition
- HTTP Methods and Status Codes

Endpoints
- Definition
- Hosting

Dependency Handling
- Dependencies File as a Blueprint

Quick Recap

- **Scope of ML**
  - Problems Solved using ML
  - Model Building Process
  - Need for Deployment
- **The Challenges**
  - Sharing and Consistency
  - Speed and Scalability
- **Model Deployment**
  - Definition
  - What It Solves For
- **Serialization**
  - Definition
  - Formats
- **API**
  - Definition
  - HTTP Methods and Status Codes
- **Endpoints**
  - Definition
  - Hosting
- **Dependency Handling**
  - Dependencies File as a Blueprint
- **Security**
  - Access Keys

Quick Recap

- **Scope of ML**
  - Problems Solved using ML
  - Model Building Process
  - Need for Deployment
- **The Challenges**
  - Sharing and Consistency
  - Speed and Scalability
- **Model Deployment**
  - Definition
  - What It Solves For
- **Serialization**
  - Definition
  - Formats
- **API**
  - Definition
  - HTTP Methods and Status Codes
- **Endpoints**
  - Definition
  - Hosting
- **Dependency Handling**
  - Dependencies File as a Blueprint
- **Security**
  - Access Keys
- **Architecture of Deployment**
  - Backend
  - Frontend

Learning Outcomes

You should now be able to:

- Summarize the need for model deployment and its role in operationalizing machine learning solutions
- Implement serialization techniques to efficiently store and transfer trained models
- Develop API endpoints to facilitate seamless interaction with deployed models
- Manage dependencies and hosting environments to ensure reliable and scalable model deployment

Power Ahead!